Fundamentos de Sistemas de Computação

Profa. Anamaria Martins Moreira
Profa. Silvana Rossetto
Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação - CCMN - UFRJ

14 de março de 2026

Sumário

1 Introdução **5**

2 Organização básica dos sistemas de computação **7**

2.1 Organização hieráquica da memória 8

2.2 O papel do sistema operacional 8

3 Números inteiros no computador **11**

3.1 Sistemas de numeração 11

3.1.1 Notação posicional 11

3.1.2 Bases usuais em computação 13

3.2 Conversões entre bases 13

3.2.1 Conversão de números de uma base qualquer para base 10 14

3.2.2 Conversão de números da base 10 para outras bases 15

3.2.3 Casos particulares de conversão - conversão entre bases onde uma é uma potência da outra 18

3.3 Operações aritméticas em diferentes bases 21

3.3.1 Adição 22

3.3.2 Subtração 23

3.3.3 Multiplicação 25

3.3.4 Divisão 27

3.3.5 Caso particular: multiplicação e divisão por potência da base 28

3.4 Números de precisão finita 29

3.4.1 Propriedades algébricas das operações aritméticas 29

3.4.2 Capacidade de representação em precisão finita 30

3.4.3 Convenções para representação finita 31

3.4.4 Sinal-magnitude 32

3.4.5 Excesso de 2^{n-1} 33

3.4.6 Complemento a 2 33

3.4.7 Operações aritméticas em Complemento a 2 35

3.4.8 Expandindo e reduzindo a representação de bits em Complemento a 2 37

4 Números reais no computador **39**

4.1 Conversão de números fracionários entre as bases 10 e 2 39

4.1.1 Conversão da base 2 para a base 10 39

4.1.2 Conversão da base 10 para a base 2 39

4.2 Padrão IEEE 754 40

4.2.1 Princípio geral da representação 41

4.2.2 Precisão simples 41

4.2.3 Precisão dupla 45

4.3 Intervalos de valores representáveis 45

4.4 Arredondamentos 46

5 Caracteres no computador **49**

5.1 Histórico e Contexto 49

5.2 Caracteres ASCII (*American Standard Code for Information Interchange*) 50

5.3 Unicode e UTF-8 52

5.3.1 UTF-8 53

6 Comunicação entre sistemas de computação **57**

6.1 Uma breve história do surgimento da Internet 57

6.1.1 Camada de Acesso à Rede 59

6.1.2 Camada Internet 62

6.1.3 Camada de Transporte 62

6.1.4 Camada de Aplicação 62

6.2 Estrutura conceitual 63

6.3 Fontes de dados e sinais 63

*

Capítulo 1

Introdução

Os **computadores** são elementos centrais da **Computação**. A Computação é uma ciência — uma área de estudos — que envolve o conjunto de conhecimentos, técnicas e metodologias referentes à construção e o uso de computadores para resolver problemas por meio do processamento automático de dados.

Um computador é uma máquina — constituída de circuitos eletrônicos — capaz de executar instruções básicas que envolvem movimentação de dados, operações lógicas e aritméticas e de desvio, como por exempo:

(i) somar dois números;
(ii) verificar se um número é zero;
(iii) mover um dado de uma parte da memória (área de armazenamento de dados do computador) para outra; e
(iv) voltar para a instrução anterior.

O conjunto de instruções básicas que um computador é capaz de realizar constitui a sua **linguagem de máquina**. Por meio dessa linguagem é que conseguimos instruir o computador a realizar a tarefa desejada. Quando um novo computador é projetado, deve-se decidir quais instruções básicas constituirão a sua linguagem de máquina. Normalmente busca-se minimizar a complexidade e o custo dos dispositivos eletrônicos empregados na construção do computador, projetando instruções simples, e ao mesmo tempo capazes de atender ao uso pretendido e às necessidades de desempenho do novo computador [Tanenbaum 1992].

Quando queremos resolver um **problema** ou realizar uma **tarefa** com o auxílio de um computador, precisamos descrever a sequência de instruções básicas que o computador deverá executar. Essa sequência de instruções é denominada **programa de computador**. Como em geral as linguagens de máquina são muito simples, escrever programas de computador usando essas linguagens torna-se um tarefa complexa e trabalhosa.

Para lidar com essa dificuldade, duas abordagens são utilizadas, ambas envolvendo a construção de um novo conjunto de instruções, de nível mais alto e mais convenientes para as pessoas utilizarem. Esse novo conjunto de instruções constitui uma nova linguagem de programação, que vamos chamar de Linguagem de nível 2 (L2), enquanto a linguagem de máquina chamaremos de Linguagem de nível 1 (L1). Na primeira abordagem — denominada **tradução** — substitui-se cada instrução do programa em L2 na sequência de instruções equivalentes em L1, gerando um novo programa em L1 que será executado pelo computador. Na segunda abordagem — denominada **interpretação** — escreve-se um programa em L1 (denominado *interpretador*) que recebe as instruções de um programa em L2 como entrada, examina uma a uma, e executa diretamente a sequência de instruções L1 correspondente. Nesse caso, não há a geração de um novo programa L1 [Tanenbaum 1992].

Seguindo essas abordagens, podemos imaginar que agora temos um *computador hipotético*,

ou uma **máquina virtual**, cuja linguagem de máquina é L2. Se tal computador pudesse ser construído a um custo suficientemente baixo e garantisse a eficiência esperada, não haveria necessidade de existir L1 ou um computador capaz de executar L1.

Independente da forma como enxergamos L1 e L2, para que essas abordagens funcionem, as linguagens L1 e L2 não podem ser muito diferentes, o que significa que, embora seja mais fácil escrever programas com a linguagem L2, ela ainda está longe de ser ideal. A solução usada é repetir a mesma ideia construindo outras linguagens (L3, L4, etc.), cada uma usando sua predecessora como base, e oferecendo abstrações de níveis mais altos para facilitar o desenvolvimento dos programas. Um computador com N níveis pode ser considerado um sistema com N máquinas virtuais, cada uma delas com a sua linguagem de máquina [Tanenbaum 1992].

Dessa forma chegamos ao conceito de sistema de computação constituído de elementos de *hardware* e camadas de *software* que funcionam juntos para executar programas/aplicações que realizam as tarefas definidas pelo programador [Bryant and O'Hallaron 2016]. Os conceitos e técnicas **fundamentais** de construção de **sistemas de computação** como uma série de níveis ou máquinas virtuais intermediárias e os detalhes de alguns desses níveis constituem o tema central da disciplina Fundamentos de Sistemas de Computação e desta apostila de apoio. As disciplinas subsequentes do curso tomarão como base os fundamentos estudados aqui e aprodundarão os estudos sobre os demais níveis e conceitos relacionados com os sistemas de computação.

Exercícios

1. Explique com suas palavras os termos:
   a) computador;
   b) linguagem de máquina;
   c) programa de computador;
   b) tradutor;
   e) interpretador;
   f) máquina virtual;
   g) sistema de computação.

2. Quais são os temas de estudo da disciplina Fundamentos de Sistemas de Computação?

Capítulo 2

Organização básica dos sistemas de computação

Um **computador** é constituído de processadores, áreas de armazenamento de dados (memórias) e dispositivos de entrada e saída que se conectam por meio de barramentos. A Figura 2.1 ilustra os componentes principais de um computador e suas interligações.

O **processador** (ou CPU — *central processing unit*) é onde a máquina interpreta, ou executa, as instruções de máquina armazenadas na memória, ou área de armazenamento principal. O processador usa um conjunto de dispositivos de armazenamento de acesso rápido, chamados **registradores**. Entre esses registradores está o *program counter* (PC) (ou ponteiro de programa) cuja função é indicar/apontar para a próxima instrução de máquina que deverá ser executada pelo processador. Do momento em que a máquina é ligada até ela ser desligada, o processador repetidamente executa a instrução apontada pelo PC e o atualiza para apontar para a próxima instrução.

A **memória principal** é o dispositivo de armazenamento temporário que armazena os programas (sequência de instruções de máquina) e os dados que esse programa manipula. A memória é organizada logicamente como um **vetor linear de bytes** (unidade básica de armazenamento), cada um com um endereço único (índice do vetor), começando em zero. As instruções de máquina que constituem o programa podem ocupar um número variado de bytes. O tamanho dos itens de dados também variam de acordo com o tipo de dado. Como exemplo, em uma máquina IA32 Linux um dado do tipo inteiro (*int*) ocupa 4 bytes, e um real grande (*double*) ocupa 8 bytes.

Os **dispositivos de E/S** (ex., mouse, teclado, monitor, disco, interface de rede) são a conexão do computador com o mundo externo e possibilitam a entrada de dados para processamento pelos computadores e a saída dos resultados obtidos para visualização das pessoas ou entrada para novos processamentos.

Os **barramentos** são condutores elétricos que carregam os dados e instruções entre os componentes. Os barramentos são normalmente projetados para transferir conjuntos de bytes de tamanho fixo, conhecidos como *words* (palavras). O número de bytes em uma palavra é um parâmetro definido pelos projetistas da máquina.

As instruções de um programa são executadas em sequência, e a execução de uma instrução envolve a realização de uma série de passos: ler a instrução da memória, interpretar a instrução, carregar os dados requeridos, executar a operação definida e atualizar o ponteiro de programa para apontar para a próxima instrução (a qual pode ou não estar em uma posição seguinte da memória em relação à instrução atual).

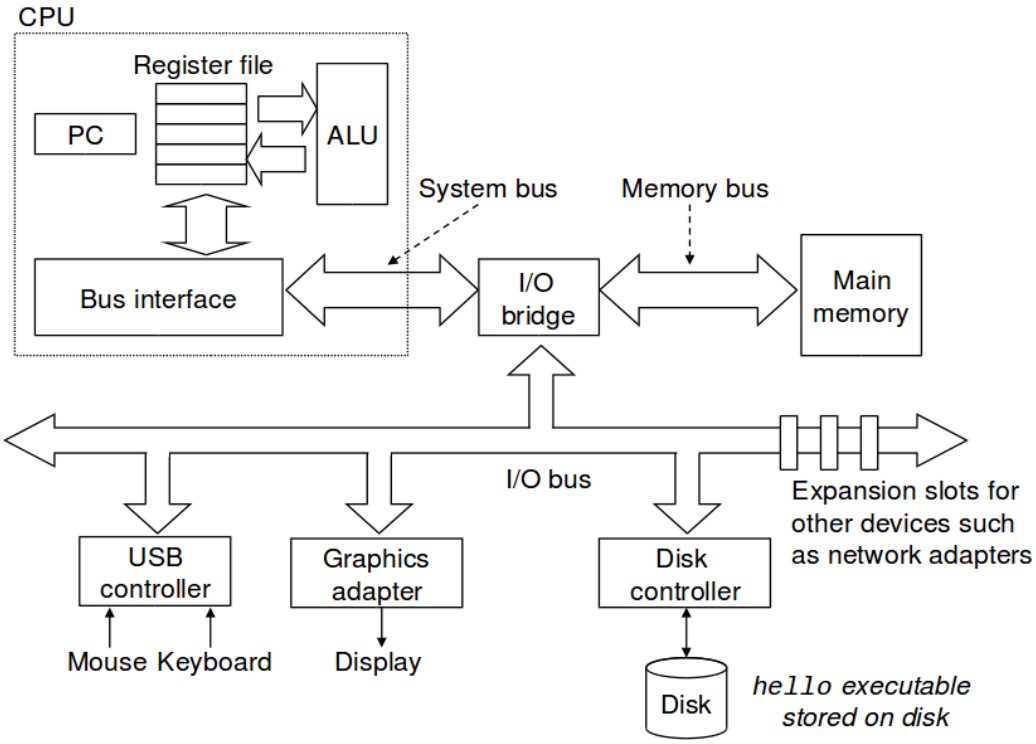

Figura 2.1. Organização típica de um computador [Bryant and O'Hallaron 2016].

2.1. Organização hieráquica da memória

Os dispositivos de memória são agrupados de forma hierárquica, como mostrado na Figura 2.2. Para lidar com a grande diferença de velocidade de operação entre o processador e a memória principal, foram acrescentados na hierarquia de memória dispositivos de armazenamento menores e mais rápidos — as memórias **cache** — que servem como áreas de armazenamento temporário de cópias dos dados que estão na memória principal.

2.2. O papel do sistema operacional

Os programas do usuário não acessam o *hardware* da máquina diretamente, ao invés disso eles fazem uso dos serviços oferecidos pelo **sistema operacional** que executa na máquina. O sistema operacional é uma camada de software interposta entre o hardware e as aplicações, como mostrado na Figura 2.3, e tem os seguintes propósitos principais:

- proteger o *hardware* de erros (ou mau uso) dos programas de usuários;
- prover mecanismos para os desenvolvedores de programas que simplifiquem o acesso ao *hardware*.
- gerenciar o uso dos recursos da máquina permitindo o seu compartilhamento por diversos programas ao mesmo tempo.

Exercícios

1. Descreva com suas palavras os seguintes componentes de um computador e sua finalidade:
   a) processador;
   b) memória principal;
   c) dispositivo de entrada e saída;
   d) barramento.

2. Por que a Figura 2.2 tem a forma de uma pirâmide?

3. Qual é a finalidade do sistema operacional em um sistema de computação?

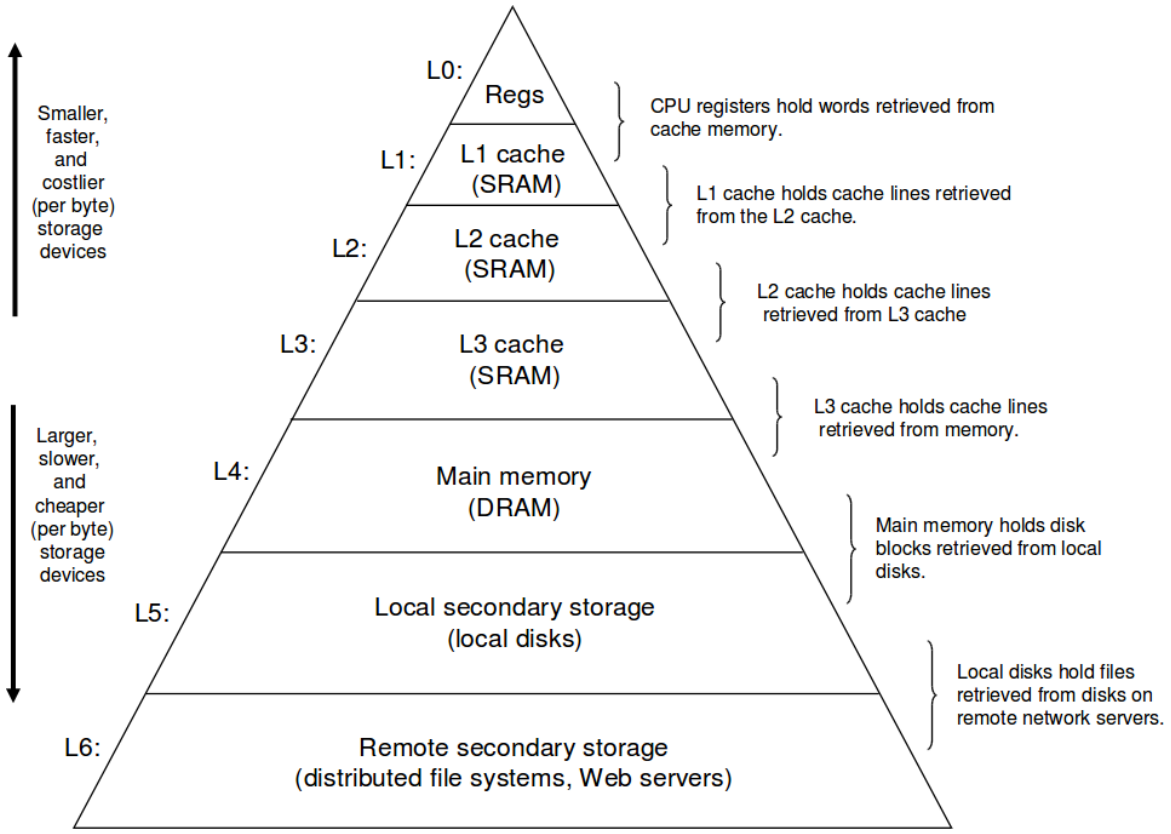

Figura 2.2. Um exemplo de hierarquia de memória [Bryant and O'Hallaron 2016].

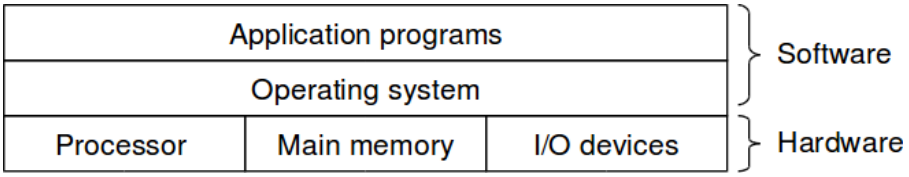

Figura 2.3. Visão em camada de um sistema de computação [Bryant and O'Hallaron 2016].

Capítulo 3

Números inteiros no computador

Neste capítulo discutiremos como os números inteiros são representados, manipulados e armazenados nos computadores. Para isso começaremos estudando sistemas de numeração (em particular o sistema de numeração binário) e, em seguida, as formas de representação e a aritmética de números inteiros positivos e negativos nos computadores. Finalizaremos o capítulo estudando operações lógicas que podem ser aplicadas no nível mais básico de representação de dados.

3.1. Sistemas de numeração

No nosso dia-a-dia usamos o **sistema de numeração decimal** com 10 algarismos (também chamados **dígitos**) distintos, de 0 a 9, para representar e manipular valores numéricos. Para usar o mesmo sistema de numeração nos computadores, seria necessário construir circuitos eletrônicos capazes de distinguir 10 níveis distintos de alguma grandeza física, como tensão elétrica. Isso tornaria os circuitos muito mais complexos e caros (na realidade, no passado, computadores trabalhando no sistema decimal já foram uma realidade, mas evoluíram!). Em razão disso, nos computadores, trabalhamos com o **sistema de numeração binário** que requer apenas 2 dígitos distintos — 0 e 1 — para representar valores numéricos. Esses dígitos são também conhecidos como **bits**, nome derivado da sua definição em inglês: *Binary Digit*.

E como usamos esses algarismos para representar os números?

Para representar qualquer informação através de bits, aplicamos convenções ou códigos que definem mapeamentos de sequências de bits em informação e vice-versa. No caso dos números, existe uma convenção que já usamos no nosso dia a dia e que está por trás de boa parte das diferentes convenções. É a notação posicional, apresentada a seguir.

3.1.1. Notação posicional

Vamos usar para exemplificar o sistema decimal, com o qual estamos mais familiarizados. Um número no sistema decimal, e aqui estamos falando também de números fracionários, e não somente de inteiros, consiste de (nessa ordem):

- uma sequência de algarismos (0 a 9) que representam a parte inteira do número, e,
- (no caso no número não ser inteiro) uma vírgula seguida de outra sequência de algarismos que representam a parte fracionária do número.

A forma geral de representação de um número decimal é mostrada abaixo:

$$d_n d_{n-1} ... d_2 d_1 d_0, d_{-1} d_{-2} d_{-3} ... d_{-k+1} d_{-k}$$

O valor representado por essa sequência de algarismos na base 10 é dado por:

$$valor = (d_n \times 10^n) + (d_{n-1} \times 10^{n-1}) + ... + (d_1 \times 10^1) + (d_0 \times 10^0) + (d_{-1} \times 10^{-1}) + ... + (d_{-k} \times 10^{-k})$$

Ou, de forma mais sintética, podemos escrever assim:

$$valor = \sum_{i=-k}^{n} d_i \times 10^i$$

Essa notação é chamada de **notação posicional**, onde a posição ocupada pelo dígito dentro da sequência determina por qual potência da base de numeração adotada o mesmo deve ser multiplicado. Dessa forma, conseguimos representar um conjunto infinito de valores usando apenas um conjunto limitado e pequeno de dígitos distintos.

Mais formalmente, a notação numérica posicional requer a definição de:

- uma *base* B, e
- um conjunto de B símbolos correspondentes a cada valor inteiro no intervalo variando de 0 a $B-1$ (os dígitos ou algarismos).

A base utilizada no sistema decimal é o valor 10, e o conjunto de símbolos é composto pelos algarismos de 0 a 9. Usualmente, quando trabalhamos com bases menores do que 10, utilizamos como símbolos o subconjunto dos algarismos de 0 até $B-1$. Para bases maiores, costuma-se utilizar as letras do alfabeto A, B, etc., respectivamente para os valores 10, 11 e assim por diante.

Com esses elementos (a base e o conjunto de dígitos) podemos construir sequências nos moldes exemplificados acima. Então, seja uma sequência de dígitos

$$s = b_i, i \in -k..n,$$

onde cada dígito b_i da sequência é um símbolo do conjunto de dígitos de uma base $B \geq 2$. Essa sequência (com $n+1+k$ símbolos) representa, em base B, o número:

$$\sum_{i=-k}^{n} b_i B^i$$

ou

$$b_{-k} \times B^{-k} + b_{-k+1} \times B^{-k+1} + ... + b_0 \times B^0 + b_1 \times B^1 + ... + b_{n-1} \times B^{n-1} + b_n \times B^n$$

Por convenção, o número é sempre representado com a posição menos significativa à direita, ou seja:

$$b_n b_{n-1}..b_1 b_0, b_{-1}..b_{-k+1} b_{-k}$$

Por exemplo, considere a sequência 347 na base 10. Sua decomposição é:

$$7 \times 10^0 + 4 \times 10^1 + 3 \times 10^2 = 7 \times 1 + 4 \times 10 + 3 \times 100 = 7 + 40 + 300 = 347$$

Agora, considere a sequência $58,25$ na base 10. Sua decomposição é:

$$5 \times 10^{-2} + 2 \times 10^{-1} + 8 \times 10^0 + 5 \times 10^1 = 5/10^2 + 2/10^1 + 8 \times 1 + 5 \times 10 = 0,05 + 0,2 + 8 + 50 = 58,25$$

Exercícios 1. Decomponha os seguintes números decimais, considerando a notação posicional. Observe que o sinal é uma informação adicional, que vem sempre a esquerda da sequência e não segue a convenção de posição/expoente.

a) 3014
b) -25,5
c) 10,01
d) -10750

3.1.2. Bases usuais em computação

As bases de numeração que nos interessam no estudo de sistemas de computação são:

- **Base 10** ou decimal (com a qual estamos familiarizados e interagimos como usuários dos computadores). Se necessário, indicamos que o número está representado em base 10 com o índice $_{10}$. Mas 10 é a nossa base de referência. Assumimos então que, se não há indicação de base, a base, por default, é 10.
  Exemplo: 345 ou 345_{10}
- **Base 2** ou binária, indicada usualmente pelo índice $_2$ ou, em programação, precedendo a sequência pelos caracteres `0B` ou `0b` (depende da linguagem).
  Exemplo: 10111_2 ou `0B10111` ou `0b10111`
- **Base 8** ou octal, indicada usualmente pelo índice $_8$ ou, em programação, precedendo a sequência pelos caracteres `0O` ou `0o` ou apenas `0` (depende da linguagem).
  Exemplo: 107_8 ou `0O107` ou `0o107` ou `0107`
- **Base 16** ou hexadecimal, indicada usualmente pelo índice $_{16}$ ou, em programação, precedendo a sequência pelos caracteres `0X` ou `0x` (depende da linguagem).
  Exemplo: $1C7_{16}$ ou `0X1C7` ou `0x1C7`

Na tabela 3.1 temos os conjuntos de símbolos usados para cada uma dessas bases, e na tabela 3.2 temos as representações nessas bases de todos os valores de 0 a 15 (zeros à esquerda podem ser accrescentados segundo a necessidade). Essas tabelinhas são bem úteis. Procure memorizá-las.

10	2	8	16
0	0	0	0
1	1	1	1
2		2	2
3		3	3
4		4	4
5		5	5
6		6	6
7		7	7
8			8
9			9
			A
			B
			C
			D
			E
			F

Tabela 3.1. Símbolos usados nas bases 2, 8, 10 e 16

Exercícios 2. Escreva os números de 0 a 20 nas seguintes bases: 2, 5, 8, 10, 11 e 16.

3.2. Conversões entre bases

A conversão de números inteiros de sua representação em uma base qualquer para a sua representação em qualquer outra base sempre pode ser feita de duas maneiras diferentes, ambas baseadas na definição da notação posicional. A primeira é simplesmente aplicar a definição da notação posicional (na base desejada), e a segunda consiste em realizar divisões sucessivas do número pela base desejada e utilizar os diferentes restos de cada divisão como os símbolos correspondentes a cada posição. O inconveniente dessas técnicas, quando estamos lidando com bases diferentes da base 10, na qual estamos

10	2	8	16
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Tabela 3.2. Valores de 0 a 15 nas bases 2, 8, 10 e 16

habituados a trabalhar, é precisar fazer cálculos em outras bases. Muitas vezes, então, optamos por usar a base 10 como uma representação intermediária. Veremos essas técnicas aplicadas às bases de mais interesse na Computação a seguir.

3.2.1. Conversão de números de uma base qualquer para base 10

Para converter números de uma base B para a base 10, basta aplicar a definição da notação posicional, fazendo os cálculos na base 10, como estamos habituados:

$$\sum_{i=-k}^{n} b_i B^i = b_{-k} \times B^{-k} + ... + b_0 \times B^0 + b_1 \times B^1 + ... + b_{n-1} \times B^{n-1} + b_n \times B^n$$

Vejamos alguns exemplos:

- $10100,1_2 =$
  $= 1 \times 2^{-1} + 0 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 + 0 \times 2^3 + 1 \times 2^4 =$
  $= 1/2 + 2^2 + 2^4 =$
  $= 0,5 + 4 + 16 =$
  $= 20,5_{10}$
- $253_8 =$
  $= 3 \times 8^0 + 5 \times 8^1 + 2 \times 8^2 =$
  $= 3 + 5 \times 8 + 2 \times 64 =$
  $= 171_{10}$
- $C3_{16} =$
  $= 3 \times 16^0 + C \times 16^1 =$
  $= 3 + 12 \times 16 =$
  $= 195_{10}$

Exercícios 3. Converta os seguintes números, nas bases indicadas, para a base 10:

a) $11011,01_2$
b) 702_8
c) BF_{16}

3.2.2. Conversão de números da base 10 para outras bases

A conversão de números inteiros de sua representação em base 10 para a sua representação em qualquer outra base pode ser feita de algumas maneiras diferentes, mas sempre baseadas na definição da notação posicional.

(A) Decomposição posicional fazendo as contas na base de destino

A primeira delas é aplicando diretamente a decomposição posicinal (como fizemos na conversão de base B para a base 10). Mas acontece aqui a dificuldade de ter que trabalhar (fazer as contas) na base destino. Vamos dar uma olhada em como fazer essas contas em outras bases mais para a frente, mas já colocamos aqui como seria.

Exemplo. Converter o número 32 da base 10 para a base 2:
$32_{10} =$
$= 2_{10} \times 10^0_{10} + 3_{10} \times 10^1_{10} =$
$= (10)_2 \times (1010)^0_2 + (11)_2 \times (1010)^1_2 =$
$= 10_2 \times 1_2 + 11_2 \times (1010)_2 =$
$= 10_2 + 11110_2 =$
$= 100000_2$

Exemplo. Converter o número 195 da base 10 para a base 16:
$195_{10} =$
$= 5_{10} \times 10^0_{10} + 9_{10} \times 10^1_{10} + 1_{10} \times 10^2_{10} =$
$= 5_{16} \times A^0_{16} + 9_{16} \times A^1_{16} + 1_{16} \times A^2_{16} =$
$= 5_{16} + 5A_{16} + 64_{16} =$
$= C3_{16}$

(B) Decomposição recursiva

Uma outra técnica consiste em realizar a seguinte sequência de passos recursivamente:

1. encontrar a maior potência (B^n) da nova base (B) menor ou igual ao número sendo convertido (num);
2. identificar quantas vezes essa potência (B^n) "cabe" no número sendo convertido, ou, mais formalmente, achar a parte inteira do quociente de num dividido por B^n. Esse será o valor do coeficiente de posição n, ou seja b_n.
3. subtrair $b_n \times B^n$ do número sendo convertido. O resultado da subtração é um novo número a ser convertido. Se esse novo número é maior ou igual à base B, retornar ao passo 1. Caso contrário, o número obtido é o coeficiente de posição zero.
4. montar a representação com os coeficientes identificados no processo em suas respectivas posições, completando eventuais coeficientes ausentes com zeros.

Exemplo. Para exemplificar, vamos converter o número 435 da base 10 para base 8:

iteração 1

- Temos que $8^2 = 64$ e $8^3 = 512$. Logo, na primeira iteração, $n = 2$.
- $435/64 = 6,7$. Logo, $b_2 = 6$.
- Sobram $435 - 6 \times 64 = 435 - 384 = 51$.
- 51 é então o novo valor a ser convertido.

iteração 2

- Temos que $8^2 = 64$ e $8^1 = 8$. Logo, a próxima potência é 8^1.
- $51/8 = 6,3$. Logo, $b_1 = 6$.
- Sobram $51 - 6 \times 8 = 51 - 48 = 3$, que é menor do que a base, sendo então o símbolo da posição 0.

Então: $435_{10} = 663_8$.

Exemplo. Vamos agora converter 435 da base 10 para a base 2.

iteração 1

- $2^8 = 256$ e $2^9 = 512$. Logo $n = 8$.
- $435/256 = 1,6....$ Logo, $b_8 = 1$.
- Sobram $435 - 1 \times 256 = 435 - 256 = 179$.

iteração 2

- $2^8 = 256$ e $2^7 = 128$. A próxima potência é 2^7.
- $179/128 = 1,3....$ Logo, $b_7 = 1$.
- Sobram $179 - 1 \times 128 = 179 - 128 = 51$.

iteração 3

- $2^6 = 64$ e $2^5 = 32$. A próxima potência é 2^5.
- $51/32 = 1,5....$ Logo, $b_5 = 1$ (observe que pulamos do b_7 para o b_5 - isso significa que b_6 vai ser zero).
- Sobram $51 - 1 \times 32 = 51 - 32 = 19$.

iteração 4

- $2^5 = 32$ e $2^4 = 16$. A próxima potência é 2^4.
- $19/16 = 1,1....$ Logo, $b_4 = 1$.
- Sobram $19 - 1 \times 16 = 19 - 16 = 3$.

iteração 5

- $2^2 = 4$ e $2^1 = 2$. A próxima potência é 2^1.
- $3/2 = 1,5$. Logo, $b_1 = 1$ (observe que pulamos do b_4 para o b_1)
- Sobram $3 - 1 \times 2 = 3 - 2 = 1$, que é menor do que a base e vai ser o símbolo da posição 0.

Preenchendo os índices ausentes (b_6, b_3 e b_2) com 0, temos que $435_{10} = 110110011_2$.

(C) Divisões sucessivas (parte inteira do número)

A segunda variante, chamada de conversão *por divisões sucessivas*, consiste em decompor o número na base de destino, usando as seguintes propriedades:

Primeira propriedade: Qualquer que seja a base B, o resto da divisão inteira de um número inteiro num por B é o valor do coeficiente b_0 ($0 =< b_0 < B$) da representação de num em base B.

Segunda propriedade: Qualquer que seja a base B, e qualquer que seja o número $num = b_n..b_1b_0$ representado em base B, se jogarmos fora o seu símbolo da posição menos significativa b_0 e deslocarmos o resto para a direita, o resultado $b_n..b_1$ é a representação do quociente inteiro de num dividido por B (vamos representar essa divisão como $div(num, B)$). Ou seja, $div(num, B) = b_1 \times B^0 + ... + b_n \times B^{n-1}$.

Como exemplo, tomando o número 9_{10} podemos decompô-lo usando a divisão inteira do número por 2 (nossa base de destino). Fazendo a primeira divisão inteira (primeira propriedade),

encontramos **quociente 4** e **resto 1**. Dessa forma, podemos reescrever o número 9 assim (*lembramos que sempre que não houver indicação de base de representação de um número, a base é 10*):

$$9 = 4 \times 2 + 1$$

ou, de outra forma:

$$9 = 4 \times 2^1 + 1 \times 2^0$$

Com essa primeira divisão, encontramos o dígito b_0 da base de destino, nesse exemplo, com valor 1.

Para encontrar o próximo coeficiente (b_1) na base de destino (B), aplicamos o mesmo procedimento sobre o **quociente** encontrado na primeira divisão, baseados no que diz a segunda propriedade. Para encontrar os demais coeficientes do número ($b_2, b_3 ... b_n$) na base de destino repetimos o processo, sempre dividindo q_i por B, obtendo como resto da divisão o coeficiente b_i e o novo quociente q_{i+1}, a ser re-dividido por B. O processo termina quando o quociente da divisão q_n é igual a zero, sendo esse último resto o coeficiente mais significativo b_n.

Voltando ao exemplo anterior, vamos fazer agora $div(4, 2)$. Daí obtemos $quociente = 2$ e $resto = 0$ e a decomposição fica assim:

$$9 = (2 \times 2^1 + 0 \times 2^0) \times 2^1 + 1 \times 2^0 =$$

$$2 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

Encontramos agora o coeficiente b_1 que é 0, e seguimos a decomposição fazendo $div(2, 2)$. Nesse novo passo, obtemos $quociente = 1$ e $resto = 0$. Voltando à decomposição, temos:

$$9 = (1 \times 2^1 + 0 \times 2^0) \times 2^2 + 0 \times 2^1 + 1 \times 2^0 =$$

$$1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

Nesse ponto, todos os coeficientes já estão na base de destino e então basta extraí-los para compor o número nessa base:

$$9_{10} = 1001_2$$

Vamos agora ver mais alguns exemplos, colocando todo o processo em uma tabela, para tornar o processo mais mecânico e assim podermos trabalhar com a conversão de números maiores.

Exemplo. Como exemplo, vamos converter 352 da base 10 para a base 8. A tabela 3.3 mostra as divisões aplicadas.

índice	dividendo	divisor	quociente	resto
0	352	8	44	0
1	44	8	5	4
2	5	8	0	5

Tabela 3.3. Divisões sucessivas de 352 por 8

Reforçando o significado da tabela 3.3, tivemos que:

$$352 = 44 \times 8 + 0 \texttt{ (linha 0)}$$

$$44 = 5 \times 8 + 4 \texttt{ (linha 1)}$$

$$5 = 0 \times 8 + 5 \texttt{ (linha 2)}$$

Substituindo o 44 por sua expansão, definida na linha 1:

$$352 = (5 \times 8 + 4) \times 8 + 0 =$$

Substituindo o 5 por sua expansão, definida na linha 2:

$$352 = ((0 \times 8 + 5) \times 8 + 4) \times 8 + 0 =$$

O que nos dá:

$$5 \times 8^2 + 4 \times 8^1 + 0 \times 8^0 = 540_8$$

Resultado: $352_{10} = 540_8$

Exemplo. Vamos converter agora 352 da base 10 para a base 2. A tabela 3.4 mostra as divisões aplicadas.

índice	dividendo	divisor	quociente	resto
0	352	2	176	0
1	176	2	88	0
2	88	2	44	0
3	44	2	22	0
4	22	2	11	0
5	11	2	5	1
6	5	2	2	1
7	2	2	1	0
8	1	2	0	1

Tabela 3.4. Divisões sucessivas de 352 por 2

Recuperando os restos de cada divisão do maior para o menor índice (da esquerda para a direita da sequência sendo construída), temos o resultado da conversão.

Resultado: $352_{10} = 101100000_2$

Exercícios 4. Escreva sua idade e o ano atual nas seguintes bases:

a) na base 10
b) na base 2
c) na base 8
d) na base 16

3.2.3. Casos particulares de conversão - conversão entre bases onde uma é uma potência da outra

Para converter de uma base para outra qualquer, sempre é possível converter para a base 10, usando a expansão acima, e em seguida da base 10 para a base final. No entanto, em alguns casos (quando uma base é uma potência da outra) pode-se evitar esse trabalho. As técnicas para as bases mais usadas em computação (2, 8 e 16) são exemplificadas a seguir.

Conversão de números em base 8 para base 2

Considere a expansão de um número octal (definição posicional) substituindo a base 8 por 2^3:

$$253_8 = 3 \times 8^0 + 5 \times 8^1 + 2 \times 8^2 =$$
$$3 \times (2^3)^0 + 5 \times (2^3)^1 + 2 \times (2^3)^2 =$$
$$3 \times 2^{(3\times 0)} + 5 \times 2^{3\times 1} + 2 \times 2^{3\times 2} =$$
$$3 \times 2^0 + 5 \times 2^3 + 2 \times 2^6$$

Nesse momento temos os valores 3, 5 e 2 multiplicando potências de 2 (0, 3 e 6, respectivamente) e teríamos vontade de colocar esses símbolos nas posições 0, 3 e 6. Isso daria algo como:

$$2005003_2$$

Mas infelizmente isso não está certo, pois 2, 5 e 3 não são símbolos válidos em uma representação binária. Precisamos ainda traduzir esses valores para suas representações binárias.

- $3 = 1 + 2 = 1 \times 2^0 + 1 \times 2^1$
- $5 = 1 + 4 = 1 \times 2^0 + 1 \times 2^2$
- $2 = 1 \times 2^1$

Voltando à expressão de onde paramos e substituindo as expansões acima, temos:

$$253_8 = 3 \times 2^0 + 5 \times 2^3 + 2 \times 2^6 =$$
$$(1 \times 2^0 + 1 \times 2^1) \times 2^0 + (1 \times 2^0 + 1 \times 2^2) \times 2^3 + (1 \times 2^1) \times 2^6 =$$
$$(1 \times 2^0 \times 2^0) + (1 \times 2^1 \times 2^0) + (1 \times 2^0 \times 2^3) + (1 \times 2^2 \times 2^3) + (1 \times 2^1 \times 2^6) =$$
$$(1 \times 2^{0+0}) + (1 \times 2^{1+0}) + (1 \times 2^{0+3}) + (1 \times 2^{2+3}) + (1 \times 2^{1+6}) =$$
$$(1 \times 2^0) + (1 \times 2^1) + (1 \times 2^3) + (1 \times 2^5) + (1 \times 2^7) = 10101011_2$$

Esse desenvolvimento trabalhoso é apenas para mostrar como chegamos no método que vamos realmente aplicar: cada símbolo da representação octal deve ser expandido em sua representação com 3 bits (com zeros à esquerda para completar os 3 bits no caso de símbolos correspondentes a valores menores do que 4).

Ou seja, sabendo que $2_8 = 010_2$, $5_8 = 101_2$ e $3_8 = 011_2$:

$253_8 =$ "010" concatenado com "101", concatenado com "011" em base 2

$253_8 = 010101011_2 = 10101011_2$ (e o zero mais à esquerda pode ser omitido, uma vez que a concatenação foi feita).

Exercícios 5. Converta os seguintes números da base 8 para a base 2:

a) 20_8
b) 7_8
c) 407_8
d) 2022_8

Conversão de números em base 16 para base 2

Como no caso da seção anterior, $16 = 2^4$ e manipulações semelhantes se aplicam, agora com grupos de 4 bits. Ou seja, vale a regra de que cada símbolo da representação hexadecimal deve ser expandido em sua representação com 4 bits (com zeros à esquerda para completar os 4 bits no caso de símbolos correspondentes a valores menores do que 8).

Vamos ver o passo a passo através de um exemplo explicado:

$$1C_{16} = C \times 16^0 + 1 \times 16^1 =$$
$$12 \times 16^0 + 1 \times 16^1 =$$
$$12 \times (2^4)^0 + 1 \times (2^4)^1 =$$
$$12 \times 2^0 + 1 \times 2^4$$

Nesse momento temos os valores 12 (C) e 1 multiplicando potências de 2 (0 e 4, respectivamente):

$$1000C_2$$

E o problema se repete com o símbolo $C_{16} = 12 = 1100_2 = 2^3+2^2$. Dado que a representação do 1 é igual na base 16 e na base 2, o símbolo 1 na posição 5, que corresponde ao valor 2^4 já está correto na representação em base 2.

Voltando à expressão de onde paramos e substituindo a expansão de C, temos:

$$1C_{16} = 12 \times 2^0 + 1 \times 2^4 =$$
$$(2^3 + 2^2) \times 2^0 + 1 \times 2^4 =$$
$$2^3 + 2^2 + 2^4 = 11100_2$$

De forma simplificada, como feito anteriormente, sabendo que $1_{16} = 1_2$ e $C_{16} = 1100_2$, então: $1C_{16} =$ "0001" concatenado com "1100" em base 2:

$$1C_{16} = 00011100_2 = 11100_2$$

Exercícios 6. Converta os seguintes números da base 16 para a base 2:

a) 20_{16}
b) 7_{16}
c) 407_{16}
d) 2022_{16}
e) $ABCD_{16}$

Conversão de números de base 2 para base 8

Pela mesma razão ($8 = 2^3$), a conversão de números de base 2 para 8 é simples. Basta agrupar os bits de 3 em 3, da direita para a esquerda, e substituir cada grupo pelo símbolo correspondente em base 8.

Exemplo.

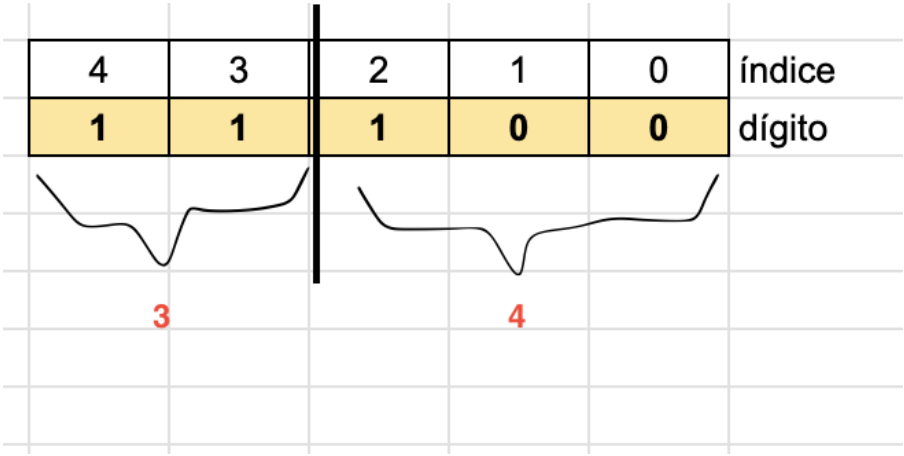

Figura 3.1. Agrupamento de bits para conversão de base 2 para base 8

11100_2 = "11" concatenado com "100" (ver Figura 3.1). Sabemos que $11_2 = 3_8$ e que $100_2 = 4_8$. Então:

$11100_2 = 34_8$

Verificando o resultado através da expansão, temos:

$11100_2 = 2^4 + 2^3 + 2^2 =$

colocando o $2^3 = 8$ em evidência e realizando a substituição, temos:

$(2^1 + 2^0) \times 2^3 + 2^2 = 3 \times 8 + 4 =$

arrumando para fazer aparecerem as posições da base 8 através de seus expoentes:

$3 \times 8^1 + 4 \times 8^0 = 34_8$

Exercícios 7. Converta os seguintes números da base 2 para a base 8:

a) 1010_2
b) 111_2
c) 10000_2
d) 10_2
e) 1101_2

Conversão de números de base 2 para base 16

Como para base 8, mas agora agrupando os bits em grupos de 4.

Exercícios 8. Converta os seguintes números da base 2 para a base 16:

a) 1010_2
b) 111_2
c) 10000_2
d) 10_2
e) 11011000_2

3.3. Operações aritméticas em diferentes bases

O procedimento genérico para realizar operações aritméticas em qualquer base é conhecer a aplicar a tabuada da operação sendo realizada na base com a qual estamos trabalhando. Fazemos isso na

base 10 sem nem mesmo pensar no que estamos fazendo, pois fomos desde cedo treinados para isso. Quando queremos trabalhar em bases de numeração diferentes da base 10, não temos em geral o automatismo dessas tabuadas. Uma solução, então, é converter os números para decimal, realizar a operação em base 10 e re-converter o resultado para a base original. Mas é também possível aplicar uma solução intermediária, que é converter os símbolos individualmente para decimal, realizar a operação parcial com os símbolos em base 10 e reconverter o resultado parcial para a base original. Nesse caso, muitas vezes conseguimos inclusive operar como se estivéssemos na base 10. Vamos ver através de exemplos, como operar em base 2, 8 ou 16.

3.3.1. Adição

Vamos começar pela operação de adição nas bases 2, 8 e 16.

Adição Binária Para fazer a adição binária sem passar pela base 10 usamos a tabuada da adição binária (tabela 3.5). Como essa tabuada é pequena, é razoável pensar em memorizá-la.

operando 1	operando 2	soma	vai um
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Tabela 3.5. Tabuada de adição de bits

Exemplo.

Vamos somar 11011_2 com 1001_2. Podemos aplicar as regras da tabuada da adição binária, da direita para a esquerda, respeitando as respectivas colunas, como fazemos com as adições na base 10. Quando há um vai um a coluna correspondente passa a ter 3 operandos, mas podemos usar as propriedades de associatividade e comutatividade da adição para somar os 3 operandos em duas etapas, cada uma com 2 operandos.

$$\begin{array}{lcccccc}
\text{(vai um)} & 1 & 1 & & 1 & 1 & \\
 & & 1 & 1 & 0 & 1 & 1_2 \\
+ & & & 1 & 0 & 0 & 1_2 \\
\hline
 & 1 & 0 & 0 & 1 & 0 & 0_2
\end{array}$$

Resultado: $11011_2 + 1001_2 = 100100_2$

Exercícios 9. Faça as seguintes operações na base 2 sem converter os números a serem somados para a base 10 (você pode converter depois de realizar o exercício para conferir o resultado!):

(a) 1101 + 010
(b) 1010 + 1010
(c) 1111 + 1111
(d) 101 + 1011

Adição Octal e Hexadecimal Nesses casos, ter a tabuada memorizada não é comum. Nos exemplos a seguir vamos usar então a técnica de converter as colunas individualmente, quando necessário. Para ajudar a fazer as traduções, temos a tabela 3.1.

Exemplo. Vamos calcular a soma $172_8 + 30_8$, sem converter os 2 números para a base 10.

$$
\begin{array}{lcccc}
\text{(vai um)} & 1 & & & \\
 & 1 & 7 & 2_8 \\
+ & & 3 & 0_8 \\
\hline
 & 2 & 2 & 2_8
\end{array}
$$

Detalhando o raciocínio da soma de cada coluna:

índice 0 — $2_8 + 0_8 = 2_8$ (aqui, como todos os valores, inclusive a soma, são os mesmos na base 8 e na base 10, conseguimos fazer a conta diretamente)

índice 1 — $7_8 + 3_8 = 7 + 3 = 10 = 8 + 2 = 1 \times 8^1 + 2 \times 8^0 = 12_8 =$ logo, na coluna fica o 2 e vai 1

índice 2 — 1_8 (do vem um) $+1_8 = 2_8$ (situação igual à do índice 0)

Resultado: $172_8 + 30_8 = 222_8$

Exemplo. Vamos calcular a soma $5D2_{16} + 5A_{16}$, sem converter os números a serem somados para a base 10.

$$
\begin{array}{lcccc}
\text{(vai um)} & 1 & & & \\
 & 5 & D & 2_{16} \\
+ & & 5 & A_{16} \\
\hline
 & 6 & 2 & C_{16}
\end{array}
$$

Detalhando o raciocínio da soma de cada coluna:

índice 0 — $2_{16} + A_{16} = C_{16}$ – aqui, como é bem simples, podemos fazer a conta sem passar pela base 10. Ou então, traduzimos o A para decimal (10), somamos 2, o que dá 12, e finalmente traduzimos de volta para hexadecimal.

índice 1 — $D_{16} + 5_{16} = 13 + 5 = 18 = 16 + 2 = 12_{16}$. Logo, na coluna atual fica o 2 e vai 1

índice 2 — 1_{16} (do vem um) $+5_{16} = 6_{16}$

Resultado: $5D2_{16} + 5A_{16} = 62C_{16}$

Exercícios 10. Faça as seguintes operações na base 8:

(a) 5 + 6
(b) 27 + 11
(c) 256 + 525

Exercícios 11. Faça as seguintes operações na base 16:

(a) 503C + 8
(b) 503C + 64

3.3.2. Subtração

Quando cada coeficiente do minuendo em cada posição i é maior ou igual ao coeficiente na mesma posição i do subtraendo, não há necessidade de "pedir ajuda" (vem um) ao coeficiente na posição $i+1$ e a subtração se faz pela subtração simples de cada coluna. Quando um coeficiente do minuendo é menor do que o coeficiente na mesma posição i do subtraendo, é necessário "pedir ajuda", ou seja, o coeficiente da posição $i+1$ é diminuído de 1 e o valor da base é adicionado ao coeficiente da posição i, passando a ser possível fazer a subtração.

minuendo	subtraendo	diferença	vem um
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Tabela 3.6. Tabuada de subtração de bits

Subtração Binária Para fazer a subtração binária sem passar pela base 10 podemos aplicar a tabuada da subtração binária (tabela 3.6).

Exemplo. Subtração sem empréstimo.

$$\begin{array}{cccccc} & 1 & 1 & 0 & 1 & 1_2 \\ - & & 1 & 0 & 0 & 1_2 \\ \hline & 1 & 0 & 0 & 1 & 0_2 \end{array}$$

Exemplo. Subtração com empréstimo. Quando temos uma situação correspondente à segunda linha da tabuada, ou seja, quando vem 1 é diferente de zero, as colunas subsequentes (mais à esquerda) são alteradas pelo empréstimo, com a subtração de 1 do valor do minuendo.

$$\begin{array}{cccccc} \text{(empréstimo)} & \text{-1} & \text{+10/-1} & \text{+10} & & \\ & 1 & 1 & 0 & 1 & 1_2 \\ - & & 1 & 1 & 0 & 1_2 \\ \hline & & 1 & 1 & 1 & 0_2 \end{array}$$

Neste exemplo, na coluna de índice 2, temos a operação $0-1$, o que implica em um empréstimo. Consequentemente, a coluna de índice 3, ao invés de ser $1-1$, passa a ser $0-1$, exigindo um novo empréstimo para a coluna de índice 4.

Exercícios 12. Faça as seguintes operações na base 2:

(a) 1101 - 100
(b) 1010 - 101
(c) 1000 - 111
(d) 10011 - 101

Subtração Octal e Hexadecimal Como na adição, não é trivial conhecer as tabuadas respectivas. Então convertemos as colunas individualmente, usando a tabela 3.1.

Exemplo. Vamos calcular a subtração $172_8 - 30_8$, sem converter os operandos para a base 10.

$$\begin{array}{cccc} & 1 & 7 & 2_8 \\ - & & 3 & 0_8 \\ \hline & 1 & 4 & 2_8 \end{array}$$

Resultado: $172_8 - 30_8 = 142_8$

Curiosidade: A subtração sem empréstimo em base 8 resulta na mesma sequência de símbolos que a mesma operação em base 10.

Exemplo. Na subtração com empréstimo, diminuímos a coluna imediatamente à esquerda da coluna que demandou o empréstimo de 1 e somamos 8 ou 10_8 ao símbolo do minuendo que precisou do empréstimo.

(empréstimo)	-1	$+10_8$	
	1	2	2_8
-		7	0_8
		3	2_8

Detalhando o raciocínio da soma de cada coluna:

índice 0 — $2_8 - 0_8 = 2_8$;
índice 1 — $2_8 < 7_8$, então pede emprestado e o minuendo fica $2_8 + 10_8 = 12_8$ (ou 10). A subtração fica então $12_8 - 7_8 = 10 - 7 = 3 = 3_8$;
índice 2 — como houve o empréstimo, o 1 da coluna mais à esquerda vira 0.

Resultado: $122_8 - 70_8 = 32_8$

Exemplo. Subtração Hexadecimal sem empréstimo.

	5	D	2_{16}
-		5	1_{16}
	5	8	1_{16}

Nesse exemplo, a única hora em que precisamos fazer as traduções para a base 10 e de volta é na hora de calcular $D_{16} - 5_{16}$, caso não tenhamos a tabuada de cabeça.

Exemplo. Exemplo Hexadecimal com empréstimo:

	5	D	2_{16}
-		5	A_{16}
	5	7	8_{16}

Detalhando o raciocínio da soma de cada coluna:

índice 0 — $2_{16} < A_{16}$; pede emprestado e fica $2_{16} + 10_{16} = 12_{16}$, daí $12_{16} - A_{16} = 8_{16}$;
índice 1 — como D_{16} emprestou 1_{16}, ficamos com $D_{16} - 1_{16} = C_{16}$, daí $C_{16} - 5_{16} = 7_{16}$;
índice 2 — $5_{16} - 0_{16} = 5_{16}$.

Resultado: $5D2_{16} - 5A_{16} = 578_{16}$

Exercícios 13. Faça as seguintes operações na base 8:

(a) 254 - 123
(b) 56 - 37

Exercícios 14. Faça as seguintes operações na base 16:

(a) 503C - 40
(b) 50EA - 503C

3.3.3. Multiplicação

Para realizar uma multiplicação em uma base qualquer, multiplicamos cada símbolo do segundo operando por cada um dos símbolos do primeiro. O resultado da multiplicação do primeiro operando pelo símbolo na posição i do segundo operando é disposto da posição i em diante (o deslocamento com o qual estamos habituados na base 10). Esse processo gera n sequências de símbolos a serem somadas, onde n é a quantidade de dígitos do segundo operando.

argumento 1	argumento 2	produto
0	0	0
0	1	0
1	0	0
1	1	1

Tabela 3.7. Tabuada de multiplicação binária

Multiplicação Binária Para realizar uma multiplicação em base 2 sem passar pela base 10 podemos aplicar a tabuada da multiplicação binária (tabela 3.7) para multiplicar cada bit do segundo operando com cada um dos bits do primeiro. Na prática, no mundo binário teremos ou uma repetição do primeiro operando (eventualmente, deslocado) ou uma linha composta integralmente de zeros (que muitas vezes nem escrevemos).

Exemplo. Vamos multiplicar $11011_2 \times 1001_2$ aplicando a tabuada de multiplicação binária.

$$\begin{array}{cccccccc}
 & & & 1 & 1 & 0 & 1 & 1_2 \\
\text{x} & & & & 1 & 0 & 0 & 1_2 \\
\hline
 & & & 1 & 1 & 0 & 1 & 1 \\
1 & 1 & 0 & 1 & 1 & & & \\
\hline
1 & 1 & 1 & 1 & 0 & 0 & 1 & 1_2
\end{array}$$

Multiplicação Octal e Hexadecimal Nas multiplicações em base octal e hexadecimal precisamos traduzir os valores da base original para 10 e vice-versa com frequência.

Exemplo. Vamos calcular $172_8 \times 30_8$.

$$\begin{array}{cccc}
 & 1 & 7 & 2_8 \\
\text{x} & & 3 & 0_8 \\
\hline
 & 0 & 0 & 0 \\
5 & 5 & 6 & \\
\hline
5 & 5 & 6 & 0_8
\end{array}$$

observando que: $3_8 \times 7_8 = 3 \times 7 = 21 = 16 + 5 = 25_8$, ou seja, ficam 5 e vão 2. Esse 2 é somado ao resultado do produto $3_8 \times 1_8 = 3_8$

Exemplo. Vamos calcular $5D2_{16} \times 51_{16}$.

$$\begin{array}{ccccc}
 & & 5 & D & 2_{16} \\
\text{x} & & & 5 & 1_{16} \\
\hline
 & & 5 & D & 2 \\
1 & D & 1 & A & \\
\hline
1 & D & 7 & 7 & 2
\end{array}$$

observando que:

- $5_{16} \times D_{16} = 5 \times 13 = 65 = 64 + 1 = 41_{16}$, ou seja, fica 1_{16} e vão 4_{16}; e
- $5_{16} \times 5_{16} = 25 = 16 + 9 = 19_{16}$, com os 4_{16}, que vieram da coluna anterior, temos $19_{16} + 4_{16} = 1D_{16}$.

Exercícios 15. Faça as seguintes operações de multiplicação na base indicada:

(a) $1010_2 \times 11_2$

(b) $1100_2 \times 101_2$
(c) $123_8 \times 43_8$
(d) $1A2_{16} \times 21_{16}$

3.3.4. Divisão

Como vimos no caso das operações anteriores, operar diretamente nas bases 8 e 16 pode ser bem trabalhoso, apesar do algoritmo geral para realizar a operação ser sempre o mesmo, independentemente da base. Em geral, optamos então pela solução de passar pela base 10. Já no caso da base 2, é importante saber trabalhar diretamente com ela, até porque, além de ser mais simples do que na base 10, são procedimentos que em alguns casos serão implementados diretamente no hardware ou nas camadas próximas a ele. Vamos ver aqui apenas exemplos para o caso particular da divisão binária.

O algoritmo de divisão binária é, como nas outras operações, similar ao da divisão decimal, com a particularidade que cada bit do quociente será 0 (o dividendo parcial era menor do que o divisor) ou 1 (o dividendo parcial era maior do que o divisor).

Exemplo. Vamos calcular $100100_2 \div 111_2$.

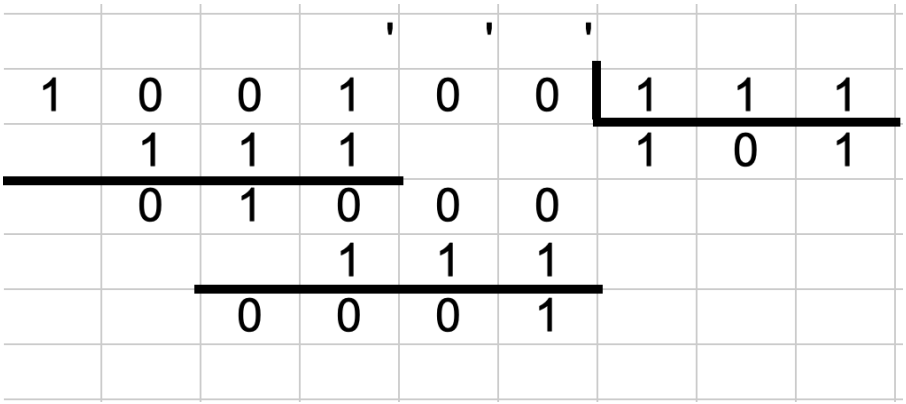

Figura 3.2. Exemplo de divisão binária

Descrevendo o que foi feito na figura 3.2:

1. Percorremos o dividendo da esquerda para a direita até obter um número maior ou igual ao divisor. Vamos chamar esse número de dividendo parcial.
2. Aí fizemos a primeira divisão, obtendo um bit 1 (bit mais significativo do quociente).
3. Subtraímos o divisor do dividendo parcial (o resultado é obrigatoriamente menor do que o divisor).
4. ”Baixamos” o próximo bit, colocando-o imediatamente à direita do resto da subtração, para construir o novo dividendo parcial.
5. Como o novo dividendo parcial é menor do que o divisor, o resultado dessa divisão é zero (um bit zero é incluído imediatamente à direita do quociente atual) e repetimos o passo 4.
6. Agora o dividendo parcial é maior do que o divisor. A nova divisão acrescenta um bit 1 no quociente. E repetimos o passo 3.
7. Como não há mais nenhum bit a baixar, o cálculo termina.

Exercícios 16. Faça as seguintes operações de divisão na base 2, fornecendo como resultado o quociente inteiro e o resto:

(a) $1010 \div 10$
(b) $10000 \div 11$

3.3.5. Caso particular: multiplicação e divisão por potência da base

As operações de multiplicação e divisão por uma potência da base (B^n) do sistema de numeração são operações que podem ser executadas de forma simples, por meio do deslocamento da vírgula que separa a parte inteira da parte fracionária do número (você pode imaginar que em um número inteiro a vírgula está implícita à direita da sequência de dígitos). Quando, em uma multiplicação por B^n, temos $n > 0$, multiplicamos o número por uma potência positiva: caminha-se com a vírgula da esquerda para a direita (acrescentando zeros no final do número, quando necessário). Quando $n < 0$, dividimos o número por uma potência positiva: caminha-se com a vírgula da direita para a esquerda. Nas seções a seguir, assumimos que $n \geq 0$, sabendo que se for menor do que zero é só inverter o sinal e a operação.

Multiplicação por potência de 2

Quando multiplicamos um número inteiro decimal por 10 (10^1), acrescentamos 0 no final do número, quando multiplicamos por 100 (10^2), acrescentamos 00 (dois zeros) no final, e assim sucessivamente.

No **sistema de numeração binário**, o mesmo pode ser feito. Por exemplo, se tomamos o número 11_2 (3 na base 2) e acrescentamos um 0 no final, ficamos com o número 110 (6 na base 2), ou seja, multiplicamos o valor inicial por 2. Outro exemplo, se tomamos o número 101_2 (5 na base 2) e acrescentamos um 0 no final, temos o número 1010_2 (10 na base 2).

Sempre que queremos multiplicar o número binário por uma potência de 2, ou seja 2^n, basta acrescentarmos n zeros no final do número.

Exercícios 17. Faça as seguintes operações de multiplicação na base 2:

(a) 111×2^1
(b) 110×2^2
(c) 11×2^3

Divisão por potência de 2

Quando dividimos um número decimal por 10 (10^1), passamos o último algarismo para a parte fracionária (resto da divisão). Se esse algarismo for 0, basta eliminá-lo. Por exemplo, $120 \div 10 = 12$. Outro exemplo, $121 \div 10 = 12,1$.

No **sistema de numeração binário**, o mesmo pode ser feito. Por exemplo, se tomamos o número 110_2 (6 na base 2) e retirarmos o 0 no final, ficamos com o número 11_2 (a representação de 3 na base 2), ou seja, dividimos o valor inicial por 2. Outro exemplo, para dividirmos o número 1000_2 (8 na base 2) por 2^2, basta retirarmos os dois últimos zeros no final, e aí temos o número 10_2 (2 na base 2).

Quando o número não termina com uma quantidade suficiente de zeros, o resultado da divisão não é um inteiro e precisamos então escrever a vírgula que estava ímplicita. Por exemplo, se fizermos $111_2 \div 2^1$ teremos $11,1_2$. (7 na base 10, dividido por 2, o resultado é 3,5). Verificando o resultado, podemos converter o número $11,1_2$ para a base 10: $11,1 = 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} = 2 + 1 + 0,5 = 3,5_{10}$.

Em resumo, sempre que queremos dividir o número binário por uma potência de 2, ou seja 2^n, basta caminharmos com a vírgula para a esquerda n vezes.

Exercícios 18. Faça as seguintes operações de divisão na base 2:

(a) $1010 \div 2^1$

(b) $1100 \div 2^2$
(c) $11000 \div 2^3$
(d) $101 \div 2^1$

3.4. Números de precisão finita

Quando executamos operações aritméticas no papel ou na cabeça, damos pouca importância à questão de quantos algarismos ou **dígitos** decimais são necessários para representar um certo número. Com os **computadores** é diferente. A quantidade de memória (ou espaço de armazenamento) disponível para representar um número é limitada e definida normalmente quando o computador é projetado. Assim, a natureza finita do computador nos força a lidar apenas com números que podem ser representados com uma quantidade fixa de dígitos. Esses números são chamados **números de precisão finita** [Tanenbaum 1992].

Nesta seção, discutiremos a representação dos números inteiros de precisão finita. Iniciaremos apresentando algumas propriedades algébricas das operações aritméticas para chamar a atenção de que nem todas serão mantidas no mundo da precisão finita. Em seguida, vamos ver como calcular a quantidade de valores diferentes que podemos representar em uma base B com precisão finita. Finalmente trataremos dos esquemas de representação de números inteiros com sinal e precisão finita e como realizar operações aritméticas nesses esquemas.

3.4.1. Propriedades algébricas das operações aritméticas

Para entender algumas das propriedades dos números de precisão finita, vamos considerar que podemos usar apenas 3 dígitos decimais para representar um múmero. Se a codificação usada é a notação posicional vista até o momento, sem ponto (ou vírgula) decimal e sem sinal, conseguimos representar apenas o conjunto dos inteiros positivos que vai de 000 a 999. Com as restrições colocadas, não conseguiremos representar:

- valores acima de 999;
- valores abaixo de 0;
- frações;
- números irracionais.

Outra consequência da limitação da quantidade de dígitos que podemos usar é que **números de precisão finita não possuem a propriedade de fechamento com relação às operações aritméticas** básicas de adição, subtração e multiplicação. A propriedade de **fechamento** da aritmética do conjunto de todos os números inteiros garante que para todo par de inteiros i e j: $i+j, i-j$ e $i \times j$ são também inteiros. Se considerarmos apenas os números inteiros positivos, a subtração já não respeita a propriedade de fechamento, como mostra o contra-exemplo $2 - 5$, que tem resultado negativo.

Voltando ao conjunto dos inteiros positivos de 3 dígitos decimais, podemos observar os seguintes exemplos:

- $900 + 500 = 1400$ (o resultado é muito grande para ser representado em 3 dígitos)
- $1 - 2 = -1$ (negativo - a representação posicional que vimos até agora não inclui informação sobre o sinal de um número)
- $100 \times 10 = 1000$ (também muito grande)

As violações encontradas são nomeadas de erro de *overflow*: os resultados encontrados estão fora da faixa de valores representáveis.

Como os computadores têm memória finita e precisam necessariamente realizar operações em números de precisão finita, os resultados de certos cálculos estarão errados do ponto de vista da Matemática clássica. No caso de *overflow*, o *hardware* é capaz de identificar e emitir um aviso.

Vamos olhar agora para outras propriedades algébricas dos números inteiros de precisão finita, para verificar se há diferenças em relação à Álgebra tradicional. Algumas propriedades que usamos com frequência, quando fazemos cálculos algébricos são a **comutatividade**, a **associatividade** e a **distributividade**.

As operações de adição e multiplicação são **operações comutativas**, ou seja, tanto faz a ordem dos operandos, o resultado é o mesmo ($a + b = b + a$ e $a \times b = b \times a$). Essa propriedade se mantém quando falamos de números inteiros de precisão finita.

A **associatividade** é outra propriedade satisfeita pela adição e pela multiplicação. Ela corresponde à equação $(a + b) + c = a + (b + c)$, para a soma e $(a \times b) \times c = a \times (b \times c)$, para a multiplicação. Quando trabalhamos com precisão finita, se o resultado final da operação é válido, ou seja, se está dentro da capacidade de representação do computador, o valor obtido continuará sendo o mesmo, independente da ordem em que a operação é executada. A diferença agora, é que, por não haver fechamento, o resultado pode ser inválido. Mas ainda assim, será inválido independentemente da ordem em que seja executada.

Finalmente, uma outra propriedade útil é a **distributividade** da multiplicação sobre a adição, expressa por $a \times (b + c) = a \times b + a \times c$. Mais uma vez, a propriedade se mantém no mundo dos inteiros com precisão finita, com a mesma ressalva que fizemos quanto à associatividade: o resultado final é o mesmo, quando o valor é válido, ou ambos serão inválidos.

Mesmo que você não se lembre dos nomes e definições dessas propriedades, você as usa regularmente. Por exemplo, associatividade e a comutatividade juntas, são o que nos permite fazer uma soma de vários números em qualquer ordem para facilitar fazer uma conta de cabeça. E a distributividade é, juntamente com as duas primeiras, usada quando colocamos fatores em evidência, por exemplo.

3.4.2. Capacidade de representação em precisão finita

Para saber o que "cabe" na representação precisamos responder à pergunta abaixo, que indica claramente que se trata de um problema de contagem.

Pergunta: Dados B símbolos diferentes (os símbolos da base B), de quantas maneiras diferentes podemos dispor de uma seleção desses símbolos em n posições distintas, numeradas de 0 a $n - 1$ (ou seja, a ordem faz diferença), sabendo-se que o mesmo símbolo pode aparecer mais de uma vez (ou seja, com repetição)?

São B^n possibilidades.

Como chegamos a essa fórmula? É um problema de contagem. Para a posição 0 temos B opções de símbolos. Para cada uma dessas opções, temos de novo B opções de símbolos para a posição 1, havendo em consequência $B \times B$ opções de preenchimento das posições 0 e 1. Da mesma maneira, para cada uma dessas B^2 opções, temos de novo B opções para preencher a posição 2, e assim por diante.

Isso nos leva então a propriedade a seguir.

Propriedade 1: Dada uma base B com seus B símbolos, uma representação em base B com n posições pode representar B^n valores diferentes.

Essa propriedade pode ser provada *por indução*, uma técnica de prova que serve para evitar que tenhamos que verificar a validade da propriedade para todos os valores possíveis de n (valores inteiros). Vamos ver aqui uma variante simples do princípio de indução para podermos provar a propriedade acima:

Indução: O princípio da indução nos naturais diz que, se:

1. uma propriedade é verdade para $n = 0$, e
2. toda vez que essa propriedade for verdade para um número natural qualquer, ela também é verdade para o próximo número natural,

então essa propriedade é verdade para todo número natural.

Matematicamente:

$$\frac{P(0) \wedge (\forall k : .P(k) \rightarrow P(k+1))}{\forall n : .P(n)}$$

A verificação do item 1 acima é chamada de *caso de base* e a do item 2, de *passo de indução*.

Aqui no nosso problema, vamos trabalhar a partir de $n = 1$, então, nosso caso de base será $n = 1$ e nossa conclusão valerá para todo $n \in N^*$.

Verificação da Propriedade 1:

1. *Caso de base ($n = 1$):* para uma representação de tamanho 1 em base B podemos representar $B = B^n$ números diferentes, correspondendo aos B símbolos da representação em base B. (ok)
2. *Passo de indução:* Supomos que para um tamanho k qualquer, a quantidade de números representáveis é B^k. Ou seja, existem B^k combinações diferentes de símbolos da base B, ocupando as posições de 0 a $k - 1$. Ao acrescentarmos a posição k, teremos B opções para preenchê-la, ou seja, cada sequência de símbolos com k posições que tínhamos dará origem a B novas sequências com $k + 1$ símbolos. Então, se tínhamos B^k sequências (hipótese acima), teremos $B \times B^k = B^{k+1}$ sequências com $k + 1$ símbolos. (ok)

   Conclusão, a propriedade 1 é válida para toda a base B e todo tamanho $n \geq 1$.

Exercícios 19.

(a) Quantos números diferentes podem ser expressos usando 2 dígitos na base 10?
(b) Quantos números diferentes podem ser expressos usando 5 dígitos na base 2?
(c) Quantos números diferentes podem ser expressos usando 32 dígitos na base 2?

3.4.3. Convenções para representação finita

Uma vez definido o tamanho da representação e, consequentemente, a quantidade de valores diferentes representáveis na base B, é também necessário definir quais serão os números efetivamente representados.

Para representar apenas números positivos ou sem sinal: Se trabalhamos apenas com números positivos, não há dúvida — poderemos representar o números na faixa que vai de 0 a $B^n - 1$ e a codificação usada nesse caso é a notação posicional como vimos até agora.

Para representar números negativos e positivos: Nesse caso, existem três convenções adotadas atualmente [Tanenbaum 1992]:

- No esquema **sinal-magnitude**, o bit (*binary digit*) mais à esquerda (o mais significativo) é usado para representar o sinal e os $n - 1$ bits restantes são usadas para representar o valor absoluto do número (como fazemos normalmente no papel).
- No esquema de **excesso de** 2^{n-1}, o número é adicionado de 2^{n-1}, deslocando seu valor absoluto para dentro das faixas de valores negativos ou positivos.
- Nos esquemas de **complementos** (complemento a 1 e complemento a 2), usa-se também um bit de sinal mais à esquerda, porém neste caso esse bit faz parte da composição do número, como detalharemos a seguir.

Espera-se encontrar um esquema de codificação com as seguintes propriedades:

(i) uma única representação para o valor zero;
(ii) mesma quantidade de números positivos e negativos. O problema é que qualquer conjunto numérico no sistema binário (que é o nosso foco) formado pelas combinações de n bits gera uma quantidade par de padrões distintos. Se apenas um deles é usado para representar o valor zero, haverá um padrão de bits a mais ou a menos para representar valores positivos e negativos [Tanenbaum 1992].

3.4.4. Sinal-magnitude

Em uma notação sinal-magnitude em base B com n posições, a posição de mais alta ordem é reservada para indicar se o número é positivo ou negativo:

$b_{n-1}\ b_{n-2}\ ...\ b_2\ b_1\ b_0$

b_{n-1} é reservado para a indicação do sinal. Logo temos para representação do valor absoluto do número apenas as $n-1$ casas restantes: $b_{n-2}\ ...\ b_2\ b_1\ b_0$

Os valores representáveis em uma notação sinal-magnitude em base B com n posições, onde a posição de mais alta ordem é reservada para indicar se o número é positivo ou negativo são, consequentemente:

- a faixa de números positivos que vai de 0 a $B^{n-1}-1$ já que temos $n-1$ dígitos para representá-los.
- a faixa de números negativos que vai de -0, ou seja, 0, até $-(B^{n-1}-1) = -B^{n-1}+1$, já que de novo temos $n-1$ dígitos para representá-los.

Então, com notação sinal-magnitude, independentemente da base que estivermos usando, poderemos representar a faixa que vai de $-(B^{n-1}-1)$ a $B^{n-1}-1$, ou seja, $2 \times B^{n-1}-1$ valores diferentes. Adicionalmente, precisa ser definido que símbolo(s) na posição b_{n-1} corresponderá(ão) ao sinal $+$ e que símbolo(s) na posição b_{n-1} corresponderá(ão) ao sinal $-$.

No caso particular da base 2, teremos:

- 2^n-2 números diferentes, na faixa $-(2^{n-1}-1)$ a $2^{n-1}-1$ e
- o bit mais significativo b_{n-1} igual a 0 significa que o número é positivo e o bit mais significativo igual a 1 significa que o número é negativo ($b_{n-1}=1$ representa o símbolo "menos", $-$).

O número 0 terá duas representações: precedido do sinal $+$ e precedido do sinal $-$ ($+0$ e -0). Essa é uma característica indesejável tanto para os projetistas de hardware quanto para programadores.

Exemplos, com $n=4$:

$1000_2 = 0000_2 = 0$

$1011_2 = -3$

$0011_2 = 3$

Exercícios 20. Converta os números decimais abaixo para a base 2 usando 5 bits e o esquema sinal-magnitude:

(a) -15
(b) -1
(c) 0
(d) 10

3.4.5. Excesso de 2^{n-1}

No esquema de **excesso de** 2^{n-1}, o número é adicionado de 2^{n-1}, deslocando seu valor absoluto para dentro das faixas de valores negativos ou positivos.

Em uma notação em excesso de 2^{n-1} em base B com n posições, a posição de mais alta ordem também indica se o número é positivo ou negativo, porém nesse caso os **números negativos são aqueles que começam com** 0, enquanto os **números positivos são aqueles que começam com** 1.

A vantagem desse esquema é que **mantem-se a ordem crescente dos valores**, isto é, as combinações iniciais (com bit mais significativo igual a 0) são usadas para os valores menores (negativos em ordem crescente) e as combinações seguintes (com bit mais significativo igual a 1) são usadas para representar os valores maiores, sempre em ordem crescente. Outra vantagem é que **temos uma única representação do valor** 0 (que ocupa a primeira combinação que começa com valor 1.

Então, com notação excesso de 2^{n-1}, independentemente da base que estivermos usando, poderemos representar a faixa que vai de $-(B^{n-1})$ a $B^{n-1} - 1$.

Considerando o caso da base 2 com $n = 4$, teremos excesso de $2^3 = 8$, logo o menor valor representável será -8 pois $-8 + 8 = 0$ (ocupa o padrão 0000). E o maior valor representável será $+7$ pois $7 + 8 = 15$ (ocupa o padrão 1111). O valor 0 será representado pelo padrão 1000, uma vez que $0 + 8 = 8$.

Vejamos outros exemplos na base 2 com $n = 4$:

$1001_2 = 1\ (9 - 8 = 1)$

$0001_2 = -7\ (1 - 8 = -7)$

$1011_2 = 3\ (11 - 8 = 3)$

$0011_2 = -5\ (3 - 8 = -5)$

Exercícios 21. Converta os números decimais abaixo para a base 2 usando 5 bits e o esquema excesso de 2^{n-1}:

(a) -15
(b) -1
(c) 0
(d) 10

3.4.6. Complemento a 2

No esquema de representação Complemento a 1 (obsoleto) adota-se a estratégia de inverter todos os bits do número para negá-lo (inverte-se cada 1 por 0 e cada 0 por 1), incluindo o bit mais a esquerda que representa o sinal (0 para números positivos e 1 para números negativos). O problema neste caso é que também teremos duas representações para o número 0.

O esquema de representação de **Complemento a 2** elimina essa questão de dupla representação do número 0 e é o **esquema adotado para representar números inteiros nos computadores modernos** (desde 1965).

Nessa representação, interpreta-se o bit mais significativo com peso negativo, e os demais com peso positivo. Veja os exemplos a seguir:

$0000 = -0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 0$
$0001 = -0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 1$
$0101 = -0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 5$

$1000 = -1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = -8$
$1011 = -1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = -5$
$1111 = -1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = -1$

Para negar um número em Complemento a 2 pode-se usar a estratégia de dois passos: primeiro faz o Complemento a 1 do número (invertendo todos os 0 por 1 e todos os 1 por 0); depois soma 1 ao resultado. Esse procedimento tem como base a observação de que a soma de um número com sua representação invertida deve ser igual a $1...1$ que em Complemento a 2 representa o valor -1. Então temos $x + \bar{x} = -1$ e daí $x + \bar{x} + 1 = 0$ ou $\bar{x} + 1 = -x$.

Representação em Complemento a 2 de números positivos e 0 (de 0 a $2^{n-1} - 1$): Idêntica à notação sinal-magnitude, sempre com o bit de sinal (bit mais significativo) igual a 0 e os bits restantes de acordo com a notação posicional. Números positivos e zero possuem exatamente a mesma representação nas duas convenções (sinal-magnitude e complemento a 2). Adicionalmente, quando o bit mais significativo é 0, tanto faz se é uma representação em complemento a 2 de n bits, sendo o mais significativo o bit de sinal, ou um valor absoluto de n bits, o número representado é o mesmo.

Representação em Complemento a 2 de números negativos (de -1 a -2^{n-1}) : Para calcular a representação em complemento a 2 de um número negativo na faixa de números representáveis faz-se:

1. converte o valor absoluto do número para representação binária com n bits.
2. inverte cada bit da representação binária obtida, ou seja, cada 1 é substituído por um 0 e cada 0 por um 1.
3. soma 1 ao resultado obtido no passo 2. Ignora o vai-um do bit mais significativo, se houver.

Exemplo. Com $n = 4$:

- $0 = 0000_2$, pois o passo 1 dá 0000, o passo 2 dá 1111 e o passo 3 dá 0000 com vai-um ignorado.
- $-3 = 1101_2$, pois o passo 1 dá 0011, o passo 2 dá 1100 e o passo 3 dá 1101.
- $3 = 0011_2$, pois é um número positivo, então não há codificação especial a fazer.

Representação Completa em Complemento a 2 com n bits Os valores representáveis em uma notação de complemento a 2 com n posições, onde a posição de mais alta ordem é reservada para indicar se o número é positivo ou negativo são:

- a faixa de números positivos que vai de 0 a $2^{n-1}-1$, já que temos $n-1$ bits para representá-los.
- a faixa de números negativos que vai de -1 até -2^{n-1}, pois, temos mais uma vez 2^{n-1} combinações diferentes, mas dessa vez estamos começando de -1, e não de 0.

Então, com notação de complemento a 2 poderemos representar a faixa de valores que vai de -2^{n-1} até $2^{n-1} - 1$ ou seja, $2 \times 2^{n-1} = 2^n$ valores diferentes (incluindo uma representação para o valor 0).

Exercícios 22. Escreva os números abaixo na base 2 usando 5 bits e a notação complemento a 2:

(a) -15
(b) -1
(c) 0
(d) 10

Exercícios 23. Transcreva os números abaixo, escritos na base 2 com notação complemento a 2 em 7 bits, para a base 10. Em outras palavras, se estamos falando a "língua" de complemento a 2, a que valores essas sequências de bits correspondem?

(a) 1111110
(b) 1000001
(c) 0111111
(d) 1010101

Inversão do sinal de um número em complemento a 2 Em alguns momentos do texto acima falamos em "negar" um número. Isso é um termo usado em computação para dizer que queremos inverter o sinal do número (também chamado de inverso aditivo). Por exemplo, o resultado de negar o número 15 seria -15 e o resultado de negar o número -15 seria 15. Quando usamos a convenção sinal-magnitude basta inverter o bit mais significativo para obter esse resultado. Quando usamos a convenção de complemento a 2 para a representação de números positivos e negativos, o processo de negação envolve o cálculo do complemento a 2 da sequência de bits original, independentemente de se o número original era negativo ou positivo.

Exemplo. Considere que estamos trabalhando em um mundo que "fala" complemento a 2 com 4 bits. Como vimos acima, o número 3 é representado pela sequência de bits 0011 e o número -3, pela sequência 1101. Para calcular a representação de -3, construímos primeiro a representação de 3, invertemos e somamos 1.

O que não vimos naquele momento é que, para inverter o sinal de -3 para 3 o processo é o mesmo. Partimos de 1101, invertemos os bits, obtendo 0010, e somamos 1, obtendo 0011, a representação do número 3.

Exercícios 24. As sequências de bits a seguir representam números em codificação binária com complemento a 2 em 7 bits. Obtenha a representação da negação (ou inverso aditivo) de cada um desses números na mesma codificação.

(a) 1111110
(b) 1000001
(c) 0111111
(d) 1010101

3.4.7. Operações aritméticas em Complemento a 2

Nesta seção, apresentamos as operações aritméticas em complemento a 2.

Adição: A adição em complemento a 2 é feita como uma operação de adição de n bits sem sinal, sem se preocupar com o fato de que o bit n de cada operando representa o sinal desse operando. A diferença ocorre apenas na interpretação do vai-um mais à esquerda da representação. Na notação de n bits sem sinal esse vai-um caracteriza um overflow. **Em complemento a 2, a situação que indica um overflow é adicionar 2 operandos de mesmo sinal e obter-se um resultado com sinal oposto.**

Vejamos alguns exemplos de adições com números representados em complemento a 2, com tamanho de representação de 8 bits.

1. dois números positivos sem *overflow*

$$
\begin{array}{cccccccccc}
 & & & 1 & 1 & & 1 & 1 & \\
 & \mathbf{0} & 0 & 0 & 1 & 1 & 0 & 1 & 1_2 \\
+ & \mathbf{0} & 1 & 0 & 0 & 1 & 0 & 0 & 1_2 \\
\hline
 & \mathbf{0} & 1 & 1 & 0 & 0 & 1 & 0 & 0_2
\end{array}
$$

2. dois números negativos sem *overflow*:

$$\begin{array}{cccccccccc}
1 & 1 & 1 & 1 & 1 & & 1 & 1 & \\
 & \mathbf{1} & 1 & 1 & 1 & 1 & 0 & 1 & 1_2 \\
+ & \mathbf{1} & 1 & 0 & 0 & 1 & 0 & 0 & 1_2 \\
\hline
 & \mathbf{1} & 1 & 0 & 0 & 0 & 1 & 0 & 0_2
\end{array}$$

   Jogamos fora o vai um mais à esquerda. Uma outra regra que é usada para identificar que não houve *overflow* é observar que houve vai um da posição $n-1$ para a posição n, mas também houve da posição $n-2$ para a posição $n-1$.

3. um número positivo e um negativo (nunca acontece *overflow*):

$$\begin{array}{cccccccccc}
 & & & 1 & 1 & & 1 & 1 & \\
 & \mathbf{1} & 1 & 0 & 0 & 1 & 0 & 0 & 1_2 \\
+ & \mathbf{0} & 0 & 0 & 1 & 1 & 0 & 1 & 1_2 \\
\hline
 & \mathbf{1} & 1 & 1 & 0 & 0 & 1 & 0 & 0_2
\end{array}$$

4. dois números negativos com *overflow*:

$$\begin{array}{cccccccccc}
1 & & & 1 & 1 & & 1 & 1 & \\
 & \mathbf{1} & 0 & 0 & 1 & 1 & 0 & 1 & 1_2 \\
+ & \mathbf{1} & 1 & 0 & 0 & 1 & 0 & 0 & 1_2 \\
\hline
 & \mathbf{0!!} & 1 & 1 & 0 & 0 & 1 & 0 & 0_2
\end{array}$$

   Somar dois números negativos nunca poderia dar como resultado um número positivo.

Subtração: A subtração $op1 - op2$ pode ser vista como a adição $op1 + (-op2)$. Ou seja, deve ser calculado o complemento a 2 do subtraendo ($op2$). A partir daí, as regras para a adição se aplicam.

Exercícios 25. Faça as operações abaixo transcrevendo os números da base 10 para a base 2, usando a representação complemento a 2 com 4 bits. Indique os casos em que ocorre *overflow* observando o bit de sinal do resultado.

(a) 5 - 3
(b) 2 - 4
(c) (-5) + (-1)
(d) 4 + 2
(e) 7 + 3
(f) (-4) + (-5)

Multiplicação ou divisão: Faz-se a multiplicação (ou divisão) dos valores absolutos dos operandos. Para isso, caso algum deles seja negativo, o seu complemento a 2 deve ser calculado para identificação do seu valor absoluto. O resultado é negativo caso os 2 operandos tenham sinais opostos e positivo caso contrário. Caso o resultado seja negativo é necessário calcular o complemento a 2 do valor absoluto obtido na operação. **No caso da multiplicação, em uma representação de tamanho n, caso o produto dos 2 valores absolutos possua o bit de posição n ou mais à esquerda igual à 1 é caracterizado um *overflow*, ou seja, resultado inválido.**

Vejamos alguns exemplos de multiplicações com números representados em complemento a 2, com tamanho de representação de 8 bits:

1. dois números positivos com *overflow*:

	0	0	0	1	1	0	1	1_2
$\times$	**0**	0	0	0	0	1	0	1_2
	1!!	1	1	1				
	0	0	0	1	1	0	1	1
+	**0**	1	1	0	1	1		
	1!!	0	0	0	0	1	1	1

2. dois números positivos sem *overflow*:

	0	0	0	0	1	0	1	1_2
$\times$	**0**	0	0	0	0	1	0	1_2
				1				
	0	0	0	0	1	0	1	1
+	**0**	0	1	0	1	1		
	0	0	1	1	0	1	1	1

3. um número positivo e um negativo com *overflow*: $\mathbf{1}0001011_2 \times \mathbf{0}0000101_2$. O primeiro operando é negativo. Logo, precisamos calcular seu valor absoluto (seu complemento a 2) para, em seguida, multiplicar:

	1	0	0	0	1	0	1	1_2
inv	**0**	1	1	1	0	1	0	0_2
+								1
	0	1	1	1	0	1	0	1_2

		0	1	1	1	0	1	0	1_2
	$\times$	**0**	0	0	0	0	1	0	1_2
1		1	1	1		1			
		0	1	1	1	0	1	0	1
	+ **1**!!	**1**!!	1	0	1	0	1		
	... !!	**0**	1	0	0	1	0	0	1

O overflow já está caracterizado mesmo antes da soma.

3.4.8. Expandindo e reduzindo a representação de bits em Complemento a 2

Algumas vezes é necessário converter o número inteiro de n bits para a sua representação com m bits, onde $m > n$, mantendo o mesmo valor. Para a notação **complemento a 2**, basta mover o bit de sinal para a posição mais a esquerda e preencher as novas posições de bit com o **valor igual ao bit de sinal**. Tomando o valor 4 como exemplo, sua representação com 4 bits em complemento a 2 é 0100, se expandirmos esse valor para 8 bits, teremos: 00000100. O valor -4 representado com 4 bits em complemento a 2 é 1100, se expandirmos esse valor para 8 bits, teremos: 11111100.

Exercícios 26.

Mostre que cada um dos valores hexadecimais abaixo em, notação complemento a 2, representa o valor -5:

(a) B
(b) FB
(c) FFB

Observe que a segunda e terceira representações podem ser derivadas da primeira com extensão do sinal.

Por outro lado, se ao invés de estender um valor com bits extras, nós reduzimos o número de bits, não será mais possível manter o mesmo valor em todos os casos. Quando truncamos um número

de n bits para um número de k bits, $k < n$, elimina-se os $n-k$ bits mais significativos. O truncamento pode alterar o valor do número (é também uma forma de *overflow*). Para um número sem sinal x, o resultado do truncamento desse número para k bits equivale a $x \bmod 2^k$ (resto da divisão inteira de x por 2^k). Por exemplo, se temos o número 1100_2 (12 na base 10) com representação de 4 bits e fazemos o truncamento para 3 bits, teremos o número 100_2 (4 na base 10) que equivale a $12 \bmod 2^3$. Se reduzirmos para 2 bits, teremos o valor final 0 ($12 \bmod 2^2$).

Capítulo 4

Números reais no computador

Como adiantamos no capítulo anterior, a notação posicional para números fracionários em base B usa o mesmo conjunto de B símbolos dos números inteiros, mas agora inclui-se o *separador decimal*, que pode ser uma *vírgula decimal* ou um *ponto decimal*, dependendo do idioma de referência. Os símbolos à direita do separador são multiplicados por potências negativas da base.

Usando o ponto decimal como separador, seja uma sequência de símbolos $s = b_i.b_j, i \in 0..m$ e $j \in -1..-n$, onde cada símbolo b_i ou b_j da sequência é um símbolo do conjunto de símbolos de uma base $B \geq 2$, essa sequência (com $n + m + 1$ símbolos) representa, em base B, o número:

$$(\sum_{i=0}^{m} b_i B^i) + (\sum_{j=-n}^{-1} b_j B^j) = b_0 \times B^0 + b_1 \times B^1 + ... + b_m \times B^m + b_{-1} \times B^{-1} + b_{-2} \times B^{-2} + ... + b_{-n} \times B^{-n}$$

4.1. Conversão de números fracionários entre as bases 10 e 2

Como com os números inteiros, a conversão de números fracionários de sua representação em base 2 (ou de qualquer outra base) para a sua representação em base 10 é feita aplicando-se diretamente a definição da notação posicional (exemplos na seção 4.1.1). Já a conversão da base 10 para outras bases não é tão imediata. Uma técnica desenvolvida para essa conversão é apresentada na seção 4.1.2.

4.1.1. Conversão da base 2 para a base 10

Aplicação direta da definição da notação posicional.

Exemplo.

$$10.1_2 = 0 \times 2^0 + 1 \times 2^1 + 1 \times 2^{-1} = 2 + 1/2 = 2,5$$

Exemplo.

$10100.011_2 = (0 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 + 0 \times 2^3 + 1 \times 2^4) + (0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}) = 2^2 + 2^4 + 2^{-2} + 2^{-3} = 4 + 16 + 1/4 + 1/8 = 20 + 0.25 + 0.125 = 20.375$

4.1.2. Conversão da base 10 para a base 2

A conversão de números fracionários de sua representação em base 10 para a sua representação em base 2 é feita convertendo a parte inteira do número com sua técnica de preferência e usando a técnica de multiplicações sucessivas descrita a seguir para a parte fracionária.

Seja n a parte fracionária do número a ser convertido:

1. multiplica n por 2.
2. seleciona a parte inteira do resultado (0 ou 1) para o próximo bit da representação binária.
3. seleciona a parte fracionária do resultado como o novo n e retorna ao passo 1. O processo termina se essa parte fracionária é zero.

Exemplo. $20.375 = 10100_2 + 0.b_{-1}b_{-2}...$

- $0.375 * 2 = 0.75$, logo $b_{-1} = 0$ e o novo n é 0.75
- $0.75 * 2 = 1.5$, logo $b_{-2} = 1$ e o novo n é 0.5
- $0.5 * 2 = 1.0$, logo $b_{-3} = 1$ e o novo n é 0
- Como o novo n, a parte fracionária de 1.0 é zero, o algoritmo termina.
- Então: $20.375 = 10100_2 + 0.b_{-1}b_{-2}b_{-3} = 10100_2 + 0.011_2 = 10100.011_2$

Exemplo. $20.3 = 10100_2 + 0.b_{-1}b_{-2}...$
(para facilitar a legibilidade, vamos omitir aqui a identificação do novo n a cada etapa)

- $0.3 * 2 = 0.6$, logo $b_{-1} = 0$
- $0.6 * 2 = 1.2$, logo $b_{-2} = 1$
- $0.2 * 2 = 0.4$, logo $b_{-3} = 0$
- $0.4 * 2 = 0.8$, logo $b_{-4} = 0$
- $0.8 * 2 = 1.6$, logo $b_{-5} = 1$
- $0.6 * 2 = 1.2$, logo $b_{-6} = 1$ ...
- Nesse caso, o algoritmo não termina nunca, já que há um ciclo. A representação binária, então, é uma aproximação do número decimal original.
- Ficamos então com a aproximação: $20.3 = 10100_2 + 0.b_{-1}b_{-2}b_{-3}b_{-4}b_{-5}b_{-6}... = 10100_2 + 0.010011_2 = 10100.010011_2$

Exercícios 1. Converta os seguintes números da base 10 para a base 2:

a) 20.5
b) 7.25
c) 0.625
d) 1.1

4.2. Padrão IEEE 754

O padrão IEEE 754 tem como objetivo definir uma representação (codificação) padronizada para os números fracionários em computadores. Ou seja, quando um programa lê na memória uma sequência de zeros e uns que tem como objetivo representar um número fracionário, ela saberá interpretá-lo corretamente. Isso não é uma questão com a qual o programador de alto nível precise normalmente se preocupar, pois os compiladores, sistemas operacionais, etc., cuidam disso para ele. Mas quem desenvolve esses outros programas, precisa.

No padrão IEEE 754 [1] são representados números normalizados e não normalizados. Como valores especiais, apresenta o zero, infinito (positivo e negativo) e NaN (Not a Number ou Não é um Número). Não vamos cuidar aqui dos não normalizados. São definidos também dois tamanhos de representação (com possibilidade de extensão): precisão simples (também conhecido como *float*) e precisão dupla (*double*).

[1] Existem duas versões do padrão, uma mais antiga, de 1985 e outra mais nova, de 2008. O que apresentamos aqui é só um básico que não mudou de uma para outra.

4.2.1. Princípio geral da representação

O princípio geral da representação se baseia na representação binária do número fracionário, como visto nas seções anteriores, e na notação científica. Nessa notação, um número binário pode ser representado como:

$$1.xxxxx_2 * 2^{yyyy}$$

onde $xxxxx$ e $yyyy$ representam sequências quaisquer de bits (tal que o resultado da operação seja igual ou aproxime o valor a ser representado). A parte $1.xxxxx_2$ da representação é chamada de *mantissa* (será representada aqui pela letra M) e o $yyyy$ é o expoente (será representado aqui pela letra E).

Essas informações necessárias para a representação de um número fracionário em notação científica binária aparecem em uma arrumação (layout) bem definida no padrão IEEE 754, em função de se é precisão simples (seção 4.2.2) ou dupla (seção 4.2.3).

4.2.2. Precisão simples

Em precisão simples são utilizados 32 bits:

- **um bit para o sinal** (0 = positivo e 1 = negativo). O bit de sinal (S) é o bit mais significativo da representação, ou seja, o bit mais à esquerda, ou bit de índice 31. Se esse bit é zero, significa que o número sendo representado é positivo e se esse bit é 1, o número é negativo.
- **8 bits para o expoente**. Os bits seguintes ao de sinal, de índice 30 a 23 são usados para a representação do expoente.
- **23 bits para a parte fracionária da mantissa**. Os bits mais à direita ou menos significativos, de índice 22 a 00, são usados para a representação da mantissa.

  Essa arrumação leva à seguinte sequência de bits:

$$SEEEEEEEEMMMMMMMMMMMMMMMMMMMMMMM$$

Atenção! Como dá para ver acima, essas sequências de bits não são muito legíveis. Por isso, usamos a facilidade de representar grupos de 4 bits com símbolos hexadecimais, agrupando bits de 4 em 4. Ao invés de 32 bits precisamos escrever apenas 8 dígitos hexadecimais.

Mas exatamente como são usados os 8 bits do expoente e os 23 bits da mantissa?

A faixa de expoentes representáveis nos 8 bits disponíveis vai de -126 a $+127$. Mas há uma particularidade: não é usada a convenção de complemento a 2 para a representação de números negativos. A convenção usada é que o expoente é representado com um deslocamento, ou seja, o que é armazenado no campo reservado para o expoente E é o valor $E + 127$, que chamaremos de exp no texto a seguir. Então, o menor expoente (-126) aparece na representação como 1 (mais precisamente, como a sequência de bits 00000001) e o maior expoente (127), como 254 (a sequência de bits 11111110). Observe que não incluímos como expoentes válidos nem -127, que seria representado como $-127 + 127 = 0$ (00000000) e nem 128, que seria representado por $128 + 127 = 255$ (11111111). A convenção IEEE 754 reserva as combinações de bits de expoente contendo apenas 0's e apenas 1's para números não normalizados, não podendo ser usadas na representação de números normalizados.

Exemplo. Imagine que os 8 bits de expoente em uma representação IEEE 754 em precisão simples sejam 00000001. Vamos decodificar essa sequência de bits e descobrir qual é o expoente correspondente.

Sabemos que esses 8 bits estão usando notação posicional (sem sinal) para representar o número $exp = E + 127$. Vamos então descobrir quem é exp.

$$exp = 00000001_2 = 2^0 = 1$$

Agora podemos calcular $E = exp - 127$

$$E = exp - 127 = 1 - 127 = -126$$

Como comentamos acima, esse é o menor expoente que podemos representar em precisão simples.

Exemplo. Imagine agora que os 8 bits sejam 11110000. Vamos decodificar essa sequência de bits e descobrir qual é o expoente correspondente.

Decodificando exp:

$$exp = 11110000_2 = 2^4 + 2^5 + 2^6 + 2^7 = 16 + 32 + 64 + 128 = 240$$

Calculando E a partir do valor de exp:

$$E = exp - 127 = 240 - 127 = 113$$

Exemplo. Imagine agora que conhecemos o expoente que queremos representar e precisamos codificá-lo seguindo o padrão IEEE 754. Vamos usar como exemplo o maior expoente possível de ser representado em precisão simples, $E = 127$.

O primeiro passo é somar 127 a ele para obter exp:

$$exp = E + 127 = 127 + 127 = 254$$

E agora precisamos codificar exp, que nesse caso significa apenas convertê-lo para a base 2 em uma convenção de 8 bits sem sinal (usando nossa técnica de preferência dentre as opções que vimos no capítulo anterior):

$$exp = 11111110$$

Essa é a sequência de bits que precisamos colocar nos bits 30 a 23 para representar o expoente $E = 127$.

E a mantissa? Como vamos representá-la?

Um detalhe importante é que apenas a parte fracionária da mantissa é armazenada, já que à esquerda do ponto decimal sempre há um 1 em notação normalizada. Uma informação que é fixa e conhecida não precisa ser armazenada.

Os 23 bits reservados para a representação da parte fracionária da mantissa (que vamos aqui representar como $frac$, usam também uma codificação posicional sem sinal, com a particularidade que o bit mais à esquerda desses 23 corresponde ao expoente -1, o segundo ao -2, e assim por diante até chegarmos ao expoente -23, no bit mais à direita (ver figura 4.1).

posição	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
potência de 2	-1	-2	-3	-4	-5	-6	-7	-8	-9	-10	-11	-12	-13	-14	-15	-16	-17	-18	-19	-20	-21	-22	-23
coeficiente (bit)	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1

Figura 4.1. Mapeamento dos bits da parte fracionária da mantissa e as potências de 2 correspondentes

Vamos ver alguns exemplos.

Exemplo. Imagine a parte fracionária da mantissa é 0. A sua codificação para os bits reservados para a parte fracionária da mantissa é uma sequência com 23 zeros. Ou seja, os 23 bits finais da representação IEEE 754 serão todos 0.

Exemplo. Se a mantissa (em base 10) é 1.25, sua parte fracionária é 0.25. Precisamos primeiramente converter esse valor para binário. Temos que $0.25 = 0.01_2$.

Então, os bits correspondentes à mantissa serão preenchidos como na figura 4.2. É importante lembrar que nesse caso estamos tratando de expoentes negativos (dígitos à direita da vírgula ou ponto decimal) e que, portanto, podemos incluir tantos ”zeros à direita” quanto forem necessários para completar o espaço reservado para o armazenamento da informação.

posição	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
potência de 2	-1	-2	-3	-4	-5	-6	-7	-8	-9	-10	-11	-12	-13	-14	-15	-16	-17	-18	-19	-20	-21	-22	-23
coeficiente (bit)	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figura 4.2. Mapeamento dos bits da parte fracionária da mantissa e as potências de 2 correspondentes

Para decodificar a mantissa de uma representação IEEE 754 de precisão simples (float), o processo é o inverso: temos inicialmente a sequência de bits que correponde a um número fracionário em binário, acrescentamos o 1 à esquerda do separador, obtendo a mantissa em binário, e depois convertemos para a base 10.

Exemplo. Consideremos a sequência de bits 11010000000000000000000. A mantissa, em binário, é 1.1101_2. Convertendo para decimal, temos $M = 1 + 1/2 + 1/4 + 1/16 = 1 + 0.5 + 0.25 + 0.0625 = 1,8125$

Agora que já sabemos codificar e decodificar cada trecho dos 32 bits do padrão, podemos juntar as informações para codificar ou decodificar o número sendo representado.

Vamos ver alguns exemplos.

Exemplo. Vamos decodificar o número que está sendo representado no padrão float com a sequência de bits

$$01000000011100000000000000000$$

- O primeiro bit é $S = 0$, diz que o número é positivo.
- os bits 30 a 23 são: $exp = 10000000 = 2^7 = 128$. Então, $E = exp - 127 = 128 - 127 = 1$
- os bits 22 a 00 são $frac = 111000000000000000$. Então, temos que $M = 1 + 0.111_2 = 1.875$
- Juntando, temos que o número representado é $1.875 \times 2^1 = 3,75$

E agora, vamos fazer o processo inverso para construir a representação de um número fracionário.

Exemplo. Vamos construir a representação float do número -4535.8125.

- O primeiro bit precisa ser $S = 1$, dizendo que o número é negativo.
- O segundo passo é obter a representação binária posicional do número. Vamos usar divisões sucessivas para a parte inteira (figura 4.3) e multiplicações sucessivas (figura 4.4) para a fracionária. A representação binária de 4535.8125 fica 1000110110111.1101.

dividendo	quociente	resto	posição
4535	2267	1	b0
2267	1133	1	b1
1133	566	1	b2
566	283	0	b3
283	141	1	b4
141	70	1	b5
70	35	0	b6
35	17	1	b7
17	8	1	b8
8	4	0	b9
4	2	0	b10
2	1	0	b11
1	0	1	b12

Figura 4.3. Divisões sucessivas para conversão de 4535 para binário

número	dobro	parte inteira	posição
0.8125	1.625	1	b-1
0.625	1.25	1	b-2
0.25	0.5	0	b-3
0.5	1	1	b-4
0	0		

Figura 4.4. Multiplicações sucessivas para conversão de 0.8125 para binário

- Agora precisamos colocar em notação científica para extrair mantissa e expoente. A mantissa tem a forma $1.\textit{alguma coisa}$. Vamos usar o fato que podemos dividir e multiplicar um número pelo mesmo valor sem alterá-lo.

$$1000110110111.1101 = 1.0001101101111101 \times 2^{12} = M \times 2^E$$

- $M = 1.0001101101111101$, então $frac = 0001101101111101$
- $E = 12$, então $exp = 139$, o que em binário com 8 bits fica 10001011.
- Juntando as diferentes partes da representação, ficamos finalmente com

$$11000101100011011011111 01...$$

a ser completado com zeros à direita para chegar até os 32 bits definidos pelo padrão.

$$11000101100011011011111010000000$$

Finalmente, usando símbolos hexadecimais para representar os bits, ficamos com:

$$C58DBE80$$

4.2.3. Precisão dupla

Em precisão dupla são utilizados 64 bits:

- **um bit para o sinal** (0=positivo e 1=negativo). O bit de sinal (S) é o bit de índice 63.
- **11 bits para o expoente**. Os bits de índice 62 a 52. O que é armazenado no campo reservado para o expoente E é o valor $E + 1023$.
- **52 bits para a parte fracionária da mantissa**. Os bits de índice 51 a 00. Apenas a parte fracionária da mantissa é armazenada.

  Essa arrumação leva à seguinte sequência de bits:

$$SEEEEEEEEEEEMMMMMMMMMMMMMMMMMMMMMMMM$$

$$MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM$$

Exemplo. Por exemplo, considere o número inteiro 2 e vejamos como é a sua representação em precisão dupla:

- O primeiro bit precisa ser $S = 0$, dizendo que o número é positivo.
- 10_2 é a representação binária de 2. Normalizando, obtemos $1.0 * 2^1$. A mantissa é então 1.0 e o expoente é 1.
- A representação do expoente 1 será $1024 = 10000000000$.
- A representação da parte fracionária da mantissa será $0...0$ (52 zeros).
- Juntando:

$$0100000000000000000000000000000000000000000000000000000000000000$$

- Ou, usando os símbolos hexadecimais para facilitar a leitura/escrita:

$$4000000000000000$$

Para pensar: Você consegue imaginar uma regra sobre a parte fracionária da mantissa na representação IEEE 754 de números inteiros que são potências de 2?

4.3. Intervalos de valores representáveis

A representação de números reais no computador na forma apresentada pelo padrão IEEE754 permite computações envolvendo números muito grandes ($|n| >> 0$) e números muito próximos de 0 ($|n| << 1$) (aproximação da aritmética de números reais). O intervalo de valores representáveis é determinado pela quantidade de dígitos no **expoente**, enquanto a precisão é determinada pela quantidade de dígitos na **fração** (mantissa).

Tabela 4.1. Resumo da codificação de valores normalizados, não-normalizados e outros

Normalizado	+/-	0<Exp<Max	qualquer padrão de bits
Não-normalizado	+/-	000...0	qualquer padrão de bits diferente de 0
Zero	+/-	000...0	0
Infinito	+/-	111...1	0
Not-a-Number	+/-	111...1	qualquer padrão de bits diferente de 0

Como discutido acima, o padrão IEEE754 trabalha com valores chamados **normalizados**, que são aqueles em que os bits do expoente são interpretados em excesso de $2^{m-1}-1$ (seu valor é $E-127$ em precisão simples e $E-1023$ em precisão dupla).

Há ainda os valores chamados **não-normalizados** que são aqueles em que o expoente assume o padrão especial em que todos os bits são igual a 0 e a mantissa armazena um valor qualquer diferente de 0. Nesse caso, é possível representar valores muito próximos de 0. **O padrão especial em que todos os bits do expoente e da mantissa são iguais a 0 é usado para representar o valor 0.** Por fim, os padrões especiais quando todos os bits do expoente são iguais a 1 são usados para representar outros valores, como infinito e números irracionais.

As Tabelas 4.1 e 4.2 mostram respectivamente os padrões usados para representar valores normalizados, não-normalizados e outros e as faixas de valores representáveis em precisão simples e dupla.

Tabela 4.2. Faixas de valores absolutos representáveis em precisão simples e dupla

Item	**Precisão simples**	**Precisão dupla**
Bits de expoente	8	11
Bits de mantissa	23	52
Expoente	excesso de 127	excesso de 1023
Intervalo expoente	-126 a +127	-1022 a +1023
Menor normalizado	$1,0 \times 2^{-126}$	$1,0 \times 2^{-1022}$
Maior normalizado	1,111..1 $\times 2^{+127}$	1,111..1 $\times 2^{+1023}$
Menor não-normalizado	$2^{-23} \times 2^{-126}$	$2^{-52} \times 2^{-1022}$
Maior não-normalizado	0,111..1 $\times 2^{-126}$	0,111..1 $\times 2^{-1022}$

4.4. Arredondamentos

O padrão IEEE-754 define diferentes modos de arredondamento quando a codificação do número extrapola o limite de bits disponíveis. O modo mais usual é o de **arredondamento para o valor mais próximo**. Voltando apra a base 10 que estamos mais acostumados, esse mode de arredondamento define que:

- até +0.4 arredonda para baixo
- a partir de +0.6 arredonda para cima
- no caso de estar no meio exato (+0.5) arredonda para cima ou para baixo para que o **dígito menos significativo seja par** (ex., 1.50 arredonda para 2.0) (2.5 arredonda para 2.0)

Na base 2, se o bit seguinte for 1 e o restante 0s, é o caso de estarmos no meio exato. Se o bit seguinte for 0, não importa os demais, arredonda-se para baixo. Se o bit seguinte for 1 e ao menos um dos demais for diferente de 0, arredenoda-se para cima.

Exemplo. Por exemplo, considere o número 0.2 e vejamos como é a sua representação em precisão simples:

- sinal = 0
- $0.2_{10} = 0.0011001100110011[0011]_2$
- normalizando obtemos $1.100110011001[1001] * 2^{-3}$
- mantissa (sem o 1 oculto): 10011001100110011001100 mais o restante $1[1001]$ (maior que metade), então arredonda para cima: 10011001100110011001101
- representação do expoente -3 será: $-3+127 = 124_{10}$ = 01111100
- representação final: 001111100 10011001100110011001101
- convertendo para hexadecimal: 3E4CCCCD

Exercícios 2. Represente os seguintes valores em ponto flutuante (padrão IEEE 754), precisão simples:

- $1,0_{10}$ (resposta: 0X3F800000)
- $-10,25_{10}$ (resposta: 0XC1240000)
- $0,125_{10}$ (resposta: 0X3E000000)
- $0,3_{10}$ (resposta: 0X3E99999A)

Exemplo. Qual valor corresponde ao número representado no padrão IEEE-754 precisão simples como: $0x40200000$?

- seu valor binário é: `{0}{100 0000 0}010 0000 0000 0000 0000 0000`
- sinal = 0 (bit mais significativo, valor positivo)
- expoente: 100 0000 0 que equivale a 128_{10}, fazendo $128 - 127$ temos que $e = 1$
- mantissa: 010 0000 0000 0000 0000 0000 equivale a **1,010 0000 ...** (bit 1 oculto)
- valor final: $1,01 \times 2^1$ que equivale a $10,1_2$ $(2,5_{10})$

Exercícios 3. Qual valor corresponde os números abaixo, representados no padrão IEEE-754 precisão simples?

- $0X40600000$ (resposta: 3,5)
- $0XC0E00000$ (resposta: -7)
- $0X3EC00000$ (resposta: 0,375)

Capítulo 5

Caracteres no computador

Neste capítulo trataremos da representação de caracteres no computador, focando em seus aspectos básicos e no padrão de codificação mais utilizado hoje em dia.

5.1. Histórico e Contexto

Programas não lidam apenas com valores numéricos. Outra importante categoria de dados a serem manipulados pelo software e, consequentemente, armazenados e manipulados pelo hardware, são textos. A base de um texto são os caracteres.

Como podemos representar os caracteres como nossas tradicionais sequências de zeros e uns?

Basicamente, o que é feito é algum tipo de mapeamento dos caracteres em números inteiros seguida da codificação desses números em bits, usando a notação posicional. O hardware, então, nem sabe que o que está armazenado em alguma célula da memória é um caractere. Alguns dos programas na hierarquia - compiladores, sistemas operacionais, montadores, etc., é que "sabem" e traduzem de volta para o usuário a sequência de bits de acordo com o tipo de informação que aquela sequência deve representar, ou seja, *decodificam* a sequência de bits.

E como é decidido qual será esse mapeamento?

Em geral, são definidos por associações e consórcios internacionais (ou nacionais de algum país forte, com reconhecimento internacional), grupos de pessoas com interesse em que a comunicação aconteça. Esses grupos definem padrões a serem respeitados pelos desenvolvedores de hardware e software para que tudo funcione como esperado [Wikipedia 2025].

Dentre os padrões ainda sendo usados hoje em dia, o mais antigo (e bem importante) é o padrão ASCII (*American Standard Code for Information Interchange*). Esse padrão usa 7 bits para representar 2^7 (128) caracteres diferentes. O foco dessa representação são caracteres usados na língua inglesa. Isso não é suficiente, por exemplo, para escrever um texto em português, que requer acentuação de alguns caracteres e o "ç". Menos ainda, para representar um texto em grego, em russo, árabe ou chinês... que possuem um repertório de caracteres completamente diferente do nosso.

Como a arquitetura dos processadores é sempre organizada em bytes (8 bits) e seus múltiplos, sobrava um bit que não estava sendo usado. Esse bit começou então a ser usado para representar outros caracteres de acordo com a necessidade dos desenvolvedores. Cada desenvolvedor criava então as chamadas "extensões" do padrão ASCII onde os códigos variando de 0 a 127 (0000 0000 a 0111 1111) continuavam compatíveis com o padrão ASCII e os códigos variando de 128 a 255 (1000 0000 a 1111 1111) representavam caracteres adicionais, específicos daquela língua, por exemplo.

Algumas dessas extensões, que ainda são usadas atualmente, são:

- **ISO-8859-1**, padrão definido pela *International Standards Association (ISO)* e também conhecida como **ISO-latin-1**,
- **Windows-1252**, projetada pela MicroSoft.

Esses padrões ainda não eram gerais o suficiente e a omnipresença da Internet trouxe muita pressão para que aplicações conseguissem se comunicar em uma maior gama de idiomas. A solução que ganhou essa disputa é o uso combinado de um código universal abstrato para representar qualquer caracter em qualquer idioma real ou fictício (até Klingon e emojis), chamado Unicode, e uma codificação desse código em uma sequência de bytes de tamanho variável, o **UTF-8**.

Veremos a seguir um pouco mais sobre as codificações ASCII e UTF-8, as duas principais codificações de caracteres.

5.2. Caracteres ASCII (*American Standard Code for Information Interchange*)

O padrão ASCII é o mais antigo dentre os que usamos atualmente. Sua primeira versão foi definida em 1963, com revisões em 1967 e 1986. Ele usa apenas 7 bits para representar caracteres, sendo capaz de representar, então, até 2^7 ou 128 caracteres diferentes.

Dentre essas sequências de bits, apenas 95 correspondem a caracteres ditos "imprimíveis" (*printable*), ou seja, caracteres que aparecem em um texto escrito. As outras 33 sequências representam os chamados caracteres de controle, sendo que alguns desses, que eram necessários no passado, não usamos mais. Um exemplo de caracter não imprimível é o chamado *bell*, que tinha como objetivo ativar uma campainha de alerta nos aparelhos de telégrafo onde o padrão foi usado originalmente. Outro exemplo, que ainda é bastante usado, é o caracter TAB, que gera um avanço da posição do cursor.

Os caracteres imprimíveis ASCII podem ser divididos em grupos. Apresentamos esses grupos a seguir, usando símbolos hexadecimais para representar as sequências de bits.

- algarismos de 0 a 9, representados pelas sequências de 30 a 39.
- letras minúsculas de a a z, representados pelas sequências de 61 a 7A.
- letras maiúsculas de A a Z, representados pelas sequências de 41 a 5A.
- o caractere espaço em branco, representado pela sequência 20.
- caracteres de pontuação e alguns símbolos matemáticos, representados pelas sequências de 21 a 2F, 3A a 40, 5B a 60 e 7B a 7E.

A correspondência entre as sequências de bits e o caracter sendo representado pode ser vista em tabelas com diferentes formatos. Na Figura 5.1 vemos todos os caracteres imprimíveis, com suas sequências de bits e suas versões numéricas em base 8, 10 e 16, lembrando que normalmente não comunicamos entre humanos diretamente as sequências de bits por questões de legibilidade. A Figura 5.2 apresenta a tabela em um formato mais compacto, bastante útil. Nela, as colunas e linhas representam respectivamente o primeiro e o segundo dígito hexadecimal correspondente à sequência de bits, e para cada par coluna/linha existem 2 células: o número decimal correspondente e o símbolo que possui aquele código. Por exemplo, a coluna 2_, linha _0, corresponde à sequência de bits 0010 0000 (20_{16}), que por sua vez seria o número 32 em decimal, e representado por essa sequência, temos o caractere espaço em branco.

Exemplo. Se sabemos que um determinado trecho da memória contém caracteres ASCII e que seu conteúdo é (em símbolos hexadecimais):

43 61 6C 43 75 6C 65 20 31 20 2B 20 35 2E

Bin	Oct	Dec	Hex	Sinal
0010 0000	040	32	20	(espaço)
0010 0001	041	33	21	!
0010 0010	042	34	22	"
0010 0011	043	35	23	#
0010 0100	044	36	24	$
0010 0101	045	37	25	%
0010 0110	046	38	26	&
0010 0111	047	39	27	'
0010 1000	050	40	28	(
0010 1001	051	41	29	)
0010 1010	052	42	2A	*
0010 1011	053	43	2B	+
0010 1100	054	44	2C	,
0010 1101	055	45	2D	-
0010 1110	056	46	2E	.
0010 1111	057	47	2F	/
0011 0000	060	48	30	0
0011 0001	061	49	31	1
0011 0010	062	50	32	2
0011 0011	063	51	33	3
0011 0100	064	52	34	4
0011 0101	065	53	35	5
0011 0110	066	54	36	6
0011 0111	067	55	37	7
0011 1000	070	56	38	8
0011 1001	071	57	39	9
0011 1010	072	58	3A	:
0011 1011	073	59	3B	;
0011 1100	074	60	3C	<
0011 1101	075	61	3D	=
0011 1110	076	62	3E	>
0011 1111	077	63	3F	?

Bin	Oct	Dec	Hex	Sinal
0100 0000	100	64	40	@
0100 0001	101	65	41	A
0100 0010	102	66	42	B
0100 0011	103	67	43	C
0100 0100	104	68	44	D
0100 0101	105	69	45	E
0100 0110	106	70	46	F
0100 0111	107	71	47	G
0100 1000	110	72	48	H
0100 1001	111	73	49	I
0100 1010	112	74	4A	J
0100 1011	113	75	4B	K
0100 1100	114	76	4C	L
0100 1101	115	77	4D	M
0100 1110	116	78	4E	N
0100 1111	117	79	4F	O
0101 0000	120	80	50	P
0101 0001	121	81	51	Q
0101 0010	122	82	52	R
0101 0011	123	83	53	S
0101 0100	124	84	54	T
0101 0101	125	85	55	U
0101 0110	126	86	56	V
0101 0111	127	87	57	W
0101 1000	130	88	58	X
0101 1001	131	89	59	Y
0101 1010	132	90	5A	Z
0101 1011	133	91	5B	[
0101 1100	134	92	5C	\
0101 1101	135	93	5D	]
0101 1110	136	94	5E	^
0101 1111	137	95	5F	_

Bin	Oct	Dec	Hex	Sinal
0110 0000	140	96	60	`
0110 0001	141	97	61	a
0110 0010	142	98	62	b
0110 0011	143	99	63	c
0110 0100	144	100	64	d
0110 0101	145	101	65	e
0110 0110	146	102	66	f
0110 0111	147	103	67	g
0110 1000	150	104	68	h
0110 1001	151	105	69	i
0110 1010	152	106	6A	j
0110 1011	153	107	6B	k
0110 1100	154	108	6C	l
0110 1101	155	109	6D	m
0110 1110	156	110	6E	n
0110 1111	157	111	6F	o
0111 0000	160	112	70	p
0111 0001	161	113	71	q
0111 0010	162	114	72	r
0111 0011	163	115	73	s
0111 0100	164	116	74	t
0111 0101	165	117	75	u
0111 0110	166	118	76	v
0111 0111	167	119	77	w
0111 1000	170	120	78	x
0111 1001	171	121	79	y
0111 1010	172	122	7A	z
0111 1011	173	123	7B	{
0111 1100	174	124	7C	|
0111 1101	175	125	7D	}
0111 1110	176	126	7E	~

Figura 5.1. Códigos ASCII dos caracteres imprimíveis. Fonte:Wikipedia

podemos decodificar o texto, usando qualquer uma das tabelas disponibilizadas aqui. Dado que 43_{16} representa o caracter `C` maiúsculo, 61_{16}, o `a` minúsculo, e assim por diante, obtemos o texto:

Calcule 1 + 5.

Observações úteis:

1. os algarismos 0 a 9 são sempre representados pela sequência que começa com 3 e a segunda parte é o algarismo: 30 representa o 0, 31 representa o 1, e asssim por diante até 39, representando o 9.
2. lembrando que as sequências de bits podem também ser interpretadas como números em notação posicional, podemos fazer contas com elas. Então, a representação de uma letra minúscula pode ser obtida a partir da representação da sua versão maiúscula somando 20_{16} ao número que corresponde à versão maiúscula da letra. Por exemplo, o código para A é 41_{16} e o para a é 61_{16}.

	0_	1_	2_		3_		4_		5_		6_		7_	
_0	não imprimíveis		32	espaço	48	0	64	@	80	P	96	`	112	p
_1			33	!	49	1	65	A	81	Q	97	a	113	q
_2			34	"	50	2	66	B	82	R	98	b	114	r
_3			35	#	51	3	67	C	83	S	99	c	115	s
_4			36	$	52	4	68	D	84	T	100	d	116	t
_5			37	%	53	5	69	E	85	U	101	e	117	u
_6			38	&	54	6	70	F	86	V	102	f	118	v
_7			39	'	55	7	71	G	87	W	103	g	119	w
_8			40	(	56	8	72	H	88	X	104	h	120	x
_9			41	)	57	9	73	I	89	Y	105	i	121	y
_A			42	*	58	:	74	J	90	Z	106	j	122	z
_B			43	+	59	;	75	K	91	[	107	k	123	{
_C			44	,	60	<	76	L	92	\	108	l	124	|
_D			45	-	61	=	77	M	93	]	109	m	125	}
_E			46	.	62	>	78	N	94	^	110	n	126	~
_F			47	/	63	?	79	O	95	_	111	o	127	DEL

Figura 5.2. Códigos ASCII dos caracteres imprimíveis em tabela compacta

5.3. Unicode e UTF-8

Unicode é um padrão desenvolvido pelo Consórcio Unicode (home.unicode.org). A proposta é ter um único conjunto de caracteres que inclui todos os caracteres de todos os idiomas escritos, e mais alguns extras, como símbolos matemáticos, emojis, idiomas inventados, etc.

Na codificação em Unicode, todo caractere é associado a um número, chamado de *code point* (a tradução seria ponto de código, mas acabamos usando o termo em inglês). Os code points são escritos como U+XXXX, onde os símbolos X são os símbolos hexadecimais (0 a F). Para os caracteres que fazem parte do padrão ASCII, o código Unicode é o mesmo número que representava o caractere em ASCII. Por exemplo, a letra `A` é U+0041 e a letra `a` é U+0061. É trabalho do consórcio Unicode definir então o que são os caracteres de cada idioma e qual o code point de cada caractere.

Mas esse não é o único mapeamento envolvido: uma vez definido o code point do caractere, precisamos ainda definir os glifos (*glyphs*) associados ao caractere, ou seja, como esse caractere vai sem apresentado ao leitor. E por outro lado, precisamos definir como os code points serão armazenados na memória. A definição de como um code point será representado na memória é um momento de codificação adicional, e é essa parte na qual vamos focar aqui.

Existem diversas codificações (*Unicode Transformation Format - UTF*) que podem ser usadas para representar code points na memória. Algumas delas são:

- UTF-32 – usa 32 bits para representar um code point
- UTF-16 – usa 16 ou 32 bits para representar um code point
- UTF-8 – usa 8, 16, 24, 32, 40 ou até 48 bits para representar um code point (é a codificação que usamos, normalmente)

A codificação UTF-32 usa quatro bytes para cada caractere (o número de bytes por caractere é fixo). Esse formato facilita a indexação de caracteres e é capaz de codificar todos os pontos de código Unicode. Em contrapartida, ocupa muito mais espaço que outras codificações e não é muito utilizado.

A codificação UTF-16 usa dois bytes (16 bits) para os primeiros 65536 code points (chamados de Plano Multilíngue Básico) e uma codificação de até 4 bytes para os caracteres adicionais.

A codificação UTF-8 é a mais utilizada nos navegadores e páginas da Internet. Usa um byte para os primeiros 128 pontos de código e até 4 bytes para os demais caracteres. Os primeiros 128 pontos de código Unicode são idênticos aos caracteres da Tabela ASCII, o que significa que qualquer texto ASCII também é um texto UTF-8.

Vamos focar no padrão UTF-8, que é o que é efetivamento usado por quase todas as aplicações atuais, em especial as aplicações que executam na Internet, por conta de ser a codificação que usa a menor quantidade possível de bytes.

5.3.1. UTF-8

A ideia por trás da codificação UTF-8 é que a quantidade de bytes a ser usada e a distribuição dos bits que definem o code point dependem do valor sendo codificado. As Tabelas 5.1 e 5.2 mostram as faixas de code points e os respectivos mapeamentos para os bits em cada byte gerado, para o caso de utilização de até 4 bytes. Elas são idênticas, mas usam convenções diferentes para indicar a correspondência entre os bits e os símbolos hexadecimais correspondentes. Como a legibilidade desse tipo de tabela não é trivial, cada um pode escolher a versão com a qual ficar mais confortável.

Tabela 5.1. Faixas de code points UTF-8 e suas respectivas codificações em bytes. Observe que foram usadas variações nos glifos dos caracteres (negrito e itálico, por exemplo), para mostrar os bits que correspondem a que cada símbolo hexadecimal.

primeiro code point	último code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+00**7**F	0**xxx**yyyy			
U+0080	U+0**7**F*F*	110**xxx**yy	10yy*zzzz*		
U+0800	U+F**F**F*F*	1110xxxx	10**yyyy**zz	10zz*aaaa*	
U+010000	U+[1](0)F**F**F*F*	11110[x](yy)	10(yy)zzzz	10**aaaa**bb	10bb*cccc*

Tabela 5.2. Faixas de code points UTF-8 e suas respectivas codificações em bytes. Observe que foram usadas cores nos glifos dos caracteres (preto, vermelho, azul e laranja), para mostrar os bits que correspondem a que cada símbolo hexadecimal.

primeiro code point	último code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0xxxyyyy			
U+0080	U+07FF	110xxxyy	10yyzzzz		
U+0800	U+FFFF	1110xxxx	10yyyyzz	10zzaaaa	
U+010000	U+[1](0)FFFF	11110[x](yy)	10(yy)zzzz	10aaaabb	10bbcccc

As duas primeiras colunas da Tabela 5.1 mostram o primeiro e último code point da faixa. Para todos os code points dentro de uma faixa, as colunas seguintes mostram como deve ser feita a codificação bit a bit. A primeira faixa (equivalente ao padrão ASCII) utiliza apenas um byte. As faixas seguintes, utilizam respectivamente dois, três e quatro bytes.

Para entender como funciona essa tabela, vamos ver alguns exemplos.

Exemplo. A palavra "Hello" em Unicode corresponde aos seguintes pontos de código (code points) [1]:

`U+0048 U+0065 U+006C U+006C U+006F.`

Então, em UTF-8, a palavra "Hello" será representada pela seguinte sequência de bytes:

`0x48 0x65 0x6C 0x6C 0x6F`

(mesma representação ASCII uma vez que todos os caracteres estão nos primeiros 128 pontos de código), ou seja, na primeira faixa de code points. Nessa faixa, todos os code points são codificados

[1] Veja em http://www.unicode.org/charts/PDF/U0000.pdf

em um único byte, que sempre começa com valor 0. Os três bits seguintes são usados para codificar o penúltimo algarismo do code point e os quatro bits restantes são usados para codificar o último algarismo do code point, como mostrado abaixo:

```
U+0048 = 0 100 1000 = 0x48
U+0065 = 0 110 0101 = 0x65
U+006C = 0 110 1100 = 0x6C
U+006F = 0 110 1111 = 0x6F
```

Exemplo. A palavra "são" em Unicode corresponde aos seguintes pontos de código (code points) [2]:

`U+0073 U+00E3 U+006F.`

Então, em UTF-8, a palavra "são" será representada pela sequência de bytes:

`0x73 0xC3 0xA3 0x6F.`

Nesse caso, o segundo ponto de código (correspondente ao **ã**) é transformado em dois bytes (segunda faixa da Tabela 5.1), como mostrado abaixo[3]:

```
U+0073 = 0 111 0011 = 0x73
U+00E3 = U+0{0}[E](3) = 110 {000} [11] 10 [10] (0011) =
                        1100 0011 1010 0011 = 0xC3 0xA3
U+006F = 0 110 1111 = 0x6F
```

Na segunda faixa, o primeiro byte sempre começa com 110. Os três bits seguintes codificam o antepenúltimo algarismo do code point, e os dois últimos bits codificam os dois bits mais significativos do penúltimo algarismo do code point. O segundo byte dessa faixa começa sempre com 10. Os dois bits seguintes codificam os dois bits menos significativos do penúltimo algarismo do code point, e os quatro bits restantes codificam o último algarismo do code point.

Exemplo. Agora vamos fazer o caminho inverso. A seguinte sequência de bytes UTF-8 (valores em hexadecimal) foi gerada ao transmitir uma mensagem de um computador para outro:

`0x41 0x6C 0xC3 0xB4 0x21.`

Será que conseguimos descobrir a sequência de code points Unicode que gerou essa sequência de bytes UTF-8 e daí "decifrar" a mensagem enviada?

Para fazer isso, vamos olhar para as quatro últimas colunas da Tabela 5.1. O dois primeiros e o último byte da sequência apresentada começam com 0, como mostrado abaixo:

```
0x41 = 0100 0001
0x6C = 0110 1100
0x21 = 0010 0001
```

Olhando para a Tabela 5.1, podemos verificar que apenas na primeira faixa algum byte pode começar em 0. Essa é a forma de deduzirmos que o code point correspondente tem que estar nessa faixa, e aí basta fazer o mapeamento direto para code points, como mostrado a seguir:

```
0x41 = 0100 0001 -> 1a linha da tabela -> U+0041
0x6C = 0110 1100 -> 1a linha da tabela -> U+006C
0x21 = 0010 0001 -> 1a linha da tabela -> U+0021
```

Sobram ainda a serem decifrados o terceiro e quarto bytes. O terceiro byte da sequência apresentada é `0xC3 = 1100 0011`, que começa com `110`. Podemos deduzir a partir dessa informação

[2] Veja em http://www.unicode.org/charts/PDF/U0000.pdf e http://www.unicode.org/charts/PDF/U0080.pdf

[3] Os símbolos de parênteses, colchetes e chaves não fazem parte do code point, estão sendo inseridos aqui apenas para facilitar a associação entre os diferentes trechos da sua representação em UTF-8.

que o code point correspondente tem que estar na segunda faixa, na qual 2 bytes são usados para representar um único code point. Consequentemente, o byte seguinte da sequência será o seu complemento, e deverá iniciar com `10`. Efetivamente, o byte seguinte é `0xB4 = 1011 0100`, que começa com `10`. Se isso não acontecesse, seria uma situação de erro, como por exemplo em um arquivo corrompido. Ficamos então com:

```
0xC3 = 110 000 11 -> 2a linha da tabela (agrega com o byte seguinte) -->
0xB4 = 10 11 0100 --> 110 {000} [11] 10 [11] (0100) -> U+0{0}[F](4)
```

ou, separando os bits de cabeçalho e concatenando os restantes:

```
0xC3 + 0xB4 = 110 00011 + 10 110100 --> (retirando os bits de cabeçalho)
--> 00011110100 --> 000 1111 0100 --> 0x0F4 --> (que leva ao code point)
--> U+00F4
```

Chegamos ao code point `U+00F4`. Juntando com os code points que já tínhamos identificado, chegamos à seguinte sequência de code points:

`U+0041 U+006C U+00F4 U+0021.`

Finalmente, buscando nas tabelas Unicode [4], temos que:
`U+0041=A;  U+006C=l;  U+00F4=ô;  U+0021=!`

E então chegamos à mensagem transmitida: *Alô!*

Exercícios 1. Decodifique a seguinte sequência de bytes UTF-8 para a sequência de code points Unicode correspondente: 0x43 0x61 0x6E 0xC3 0xA7 0xC3 0xA3 0x6F. Depois verifique nas tabelas Unicode qual foi a mensagem original.

Exercícios 2. Converta a expressão *¡Chão!* para a sequência de bytes UTF-8 correspondente (escreva os resultados em hexadecimal). Os pontos de código Unicode usados na expressão são: ¡ (U+00A1), C (U+0043), h (U+0068), ã (U+00E3), o (U+006F), ! (U+0021).

[4] Veja em http://www.unicode.org/charts/PDF/U0000.pdf e http://www.unicode.org/charts/PDF/U0080.pdf

Capítulo 6

Comunicação entre sistemas de computação

Até o momento, no nosso curso, estudamos como os sistemas de computação funcionam de forma geral e, com um pouco mais de profundidade, como dados são armazenados nesses sistemas usando a representação binária. No entanto, ainda não exploramos um aspecto também fundamental: como um sistema de computação se comunica com outro.

Abordaremos neste capítulo, de forma sucinta, a comunicação de dados entre sistemas de computação, apresentando conceitos básicos que nos permitem ter uma visão geral de como um sistema de comunicação digital funciona. Começaremos apresentando brevemente a Internet — a rede de comunicação que interconecta sistemas de computação no mundo todo — seu histórico e estrutura geral. Depois abordaremos aspectos conceituais sobre como se dá a troca de dados entre sistemas de computação por meio de uma rede de comunicação.

6.1. Uma breve história do surgimento da Internet

A pergunta que norteia nosso estudo agora é: como os sistemas de computação são capazes de se comunicar entre si, a ponto de, hoje em dia, nossa sociedade estar completamente dependente da interconexão entre eles? A comunicação entre sistemas de computação possibilitou o surgimento e a consolidação de diversos serviços e aplicações indispensáveis na sociedade contemporânea. Exemplos disso incluem o comércio eletrônico, serviços bancários online, plataformas de *streaming* de vídeo e música, redes sociais, sistemas de mensagens instantâneas, além de serviços críticos como monitoramento de infraestruturas, saúde conectada e transporte inteligente.

Porém, para que possamos compreender como, hoje, vivemos em uma sociedade cuja maior parte das atividades depende da interconexão entre sistemas de computação, é necessário voltar no tempo, mais precisamente para a década de 1950. Naquele período, o mundo estava no meio da Guerra Fria, quando os Estados Unidos e a União Soviética disputavam a hegemonia mundial. Dentro desse contexto, o Departamento de Defesa dos Estados Unidos (DoD — *Department of Defense*) estava preocupado com a resiliência de seus canais de comunicação em caso de uma guerra nuclear. Naquela época, a comunicação militar dos Estados Unidos era baseada na *Rede Pública de Telefones*, porém ela era considerada vulnerável.

A Rede Pública de Telefones era considerada vulnerável porque, basicamente, para que uma ligação telefônica fosse realizada, era necessário estabelecer um *link*, ou uma conexão direta, entre os dois telefones que iriam se comunicar (conceitualmente chamado de *comutação por circuito*). Ou seja, havia um enlace físico sendo criado entre os dois terminais. Assim, ao atacar ou danificar algumas das centrais telefônicas, seria possível interromper o funcionamento da rede: bastava comprometer pontos estratégicos para dividir a infraestrutura em partes isoladas. Dessa forma, ao quebrar a ligação entre duas partes da rede, ela se fragmentava, impossibilitando a comunicação entre os terminais afetados. Por conta disso, surgiu a necessidade de criar uma rede mais robusta e resiliente a

ataques, em que, caso uma parte da rede fosse comprometida, a comunicação ainda pudesse fluir por outros caminhos. O tratamento dessa demanda foi uma das principais motivações para as pesquisas que levaram ao surgimento da ARPANET (*Advanced Research Projects Agency Network* - ou Rede da Agência de Projetos de Pesquisa Avançada).

A ARPANET surgiu em 1969 com o objetivo de interligar diferentes instituições por meio de linhas telefônicas de 56 kbps. Contudo, havia uma diferença fundamental em relação ao modelo de **comutação por circuito** que predominava até então. Dessa vez, a rede consistia em alguns mini-computadores interconectados por linhas telefônicas, mas que eram capazes de realizar **comutação por pacotes**. Ou seja, em vez de se criar um link direto e exclusivo entre dois computadores sempre que fosse necessário transmitir uma mensagem, o processo passava a ser diferente: **a mensagem era dividida em pequenos pacotes, e esses pacotes eram então enviados pela rede**.

Essa mudança fundamental — da comutação por circuitos para a comutação por pacotes — permitiu que a rede se tornasse mais tolerante a falhas, conforme havia sido requisitado anteriormente pelo Departamento de Defesa. Isso ocorre porque, com a comutação por pacotes, a informação não depende mais de um circuito direto e exclusivo: se esse circuito fosse rompido, como no modelo anterior, a comunicação seria interrompida. Agora, como a mensagem é dividida em pacotes, os pedaços da informação podem ser transmitidos por caminhos independentes. Com isso, caso ocorra um problema na rede, seja por falha ou ataque, os pacotes podem ser reenviados por outros caminhos. Isso é possível porque os nós que participam da rede não apenas enviam e recebem mensagens, mas também podem encaminhar pacotes de outros dispositivos para diferentes destinos, funcionando como **roteadores** dentro da rede.

O projeto foi tão bem-sucedido que, em pouco tempo, outras instituições e universidades começaram a querer se juntar à ARPANET. Contudo, nesse momento, surgiu um novo desafio: como interconectar agora centenas de universidades e instituições governamentais por meio de links que não fossem apenas as tradicionais linhas telefônicas de 56 kbps, mas também links via satélite ou rádio? Além disso, outro problema importante se apresentava: como interconectar diferentes redes, que poderiam ter tecnologias distintas de comunicação?

Diante disso, surgiu a necessidade de desenvolver uma arquitetura de rede capaz de realizar essa interconexão de forma flexível e resiliente. Nesse esforço, liderado pelas pessoas envolvidas na implementação dessas redes na época, foi concebido aquilo que, posteriormente, viria a ser conhecido como o **modelo de referência TCP/IP** (Transmission Control Protocol/Internet Protocol), que é a base para o funcionamento da Internet como conhecemos hoje.

O modelo TCP/IP precisava ser flexível o suficiente para que diferentes meios de comunicação — como linhas telefônicas, conexões via satélite e enlaces de rádio — pudessem ser utilizados. Além disso, ele precisava ser capaz de suportar o crescimento de demanda à medida que mais redes fossem adicionadas ao sistema. Outro requisito essencial era a resiliência: caso ocorresse um problema em alguma parte da rede, como tanto temia o Departamento de Defesa, a transmissão deveria ser capaz de seguir por outros caminhos, garantindo a continuidade da comunicação. Dessa forma, o modelo TCP/IP foi concebido com os **princípios de flexibilidade, escalabilidade e resiliência**.

Para que esses princípios — flexibilidade, escalabilidade e resiliência — fossem alcançados, era necessário que as diferentes responsabilidades fossem divididas e que os papéis fossem muito bem definidos. Afinal, como permitir que um dispositivo que funciona utilizando uma linha telefônica seja capaz de se comunicar com outro dispositivo conectado por um link via satélite? Como possibilitar que um dispositivo seja capaz de endereçar outro, localizado em uma rede diferente, sem que precise saber como chegar até ele? Além disso, como garantir que os dados enviados não se percam ao longo do caminho e que sejam recebidos de forma correta nos sistemas de computação de destino? Por fim, como permitir que diferentes aplicações, executando em dispositivos distintos, possam se conectar e trocar dados, abstraindo completamente os diferentes caminhos e tipos de enlace que podem ser

utilizados ao longo da rede de comunicação?

Essas e outras perguntas que surgiram naquela época começaram, progressivamente, a ser respondidas por meio de um modelo que dividiu a tarefa de realizar essa interconexão em quatro camadas. A primeira camada foi chamada de **Enlace**; a segunda, de **Internet**; a terceira, de **Transporte**; e, por fim, a quarta e última camada foi chamada de **Aplicação**. Com isso, basicamente, o que o modelo TCP/IP fez foi dividir as responsabilidades da comunicação entre os próprios sistemas de computação da seguinte forma: a tarefa de permitir que um computador se conecte a outro na mesma rede ficou sob a responsabilidade da camada de Enlace. Quando fosse necessário estabelecer a conexão com outras redes, essa tarefa passaria a ser função da camada chamada Internet. Caso fosse necessário transportar os dados de forma que garantisse a conexão confiável (ou não) entre os computadores, essa função seria atribuída à camada de Transporte. Por fim, as diferentes aplicações poderiam se preocupar apenas com a manipulação dos seus dados, requisitando da camada de Transporte o serviço desejado, que poderia ou não oferecer garantia de entrega.

Nº	**Camada**	**Função principal**
4	Aplicação	Permite a comunicação entre aplicações, abstraindo detalhes da rede.
3	Transporte	Garante a entrega dos dados entre processos, podendo ser confiável ou não.
2	Internet	Responsável pelo endereçamento e roteamento entre redes diferentes.
1	Link (Enlace)	Define como os dados são transmitidos fisicamente dentro de uma mesma rede.

Tabela 6.1. Camadas do modelo TCP/IP e suas funções principais

Essa divisão de responsabilidades foi tão bem-sucedida que se tornou a base de como computadores e dispositivos se interconectam através da Internet até os dias de hoje. Com o passar do tempo, as antigas linhas telefônicas de 56 kbps deixaram de ser utilizadas, dando espaço a links com maior velocidade, como, por exemplo, as conexões de banda larga. Tecnologias como o 4G e, mais recentemente, o 5G surgiram, aproveitando os desenvolvimentos posteriores da telefonia celular.

Atualmente, uma fibra óptica pode chegar até a casa das pessoas, permitindo que uma família comum tenha mais banda do que toda a rede disponível na época em que a ARPANET foi concebida. Com a chegada da Internet das Coisas (IoT), há uma quantidade massiva de dados sendo gerados e coletados, provenientes tanto de redes sociais quanto de dispositivos comuns. Além disso, o avanço da Inteligência Artificial e a mobilidade de telefones e computadores mostram como a divisão em camadas, pensada lá nas décadas de 1960 e 1970, foi robusta e flexível o suficiente para servir de alicerce a um mundo que, na época de sua concepção, seria considerado ficção científica.

Dessa forma, este capítulo está dividido em quatro partes, cada uma dedicada a uma das camadas do modelo TCP/IP. Em cada parte, vamos aprender sobre os desafios enfrentados e as tecnologias desenvolvidas para superá-los, bem como entender como cada uma dessas camadas se relaciona com a rede de computadores que utilizamos atualmente.

6.1.1. Camada de Acesso à Rede

A camada de Acesso à Rede do modelo TCP/IP será estudada de forma dividida em duas camadas conceituais, a fim de facilitar a compreensão de suas funções e das tecnologias envolvidas.

Camada física

Toda comunicação em rede começa com uma conexão física a uma rede local. Seja por meio de um computador, celular ou qualquer dispositivo conectado, a transmissão dos dados depende inevitavelmente de um meio físico. A conexão pode acontecer com fios (por meio de cabos de cobre ou fibra óptica, por exemplo) ou de forma sem fio (através de ondas eletromagnéticas). Dessa forma, um dos primeiros desafios é permitir que as informações sejam enviadas no meio físico utilizado para se realizar a conexão em sua rede local.

A camada física é responsável pela transmissão de bits através dos diferentes tipos de meio, como cabos, ondas eletromagnéticas e fibras ópticas. Dessa forma, definir como a mensagem deve ser enviada e transmitir os bits de um dispositivo a um outro dispositivo na mesma rede é a principal tarefa da camada física. Para isso, os padrões definidos para a camada física especificam basicamente: (i) como fazer a codificação da mensagem, (ii) como sinalizar a mensagem e (iii) os componentes físicos.

Codificação da mensagem

Codificação é a conversão de um fluxo de bits em um ”código”pré-definido. Um código é um padrão no qual os bits são organizados. Com um código definido entre o transmissor e o receptor, permite-se que ambos interpretem corretamente os dados. Na figura 6.1 podemos ver: (a) o stream de bits inicial; (b) NRZ (Non-Return-to-Zero); (c) NRZI (Non-Return-to-Zero invertido); (d) Manchester e (e) Bipolar encoding.

A codificação (b) NRZ (Non-Return-to-Zero) é a mais simples. Nela, o bit 1 é representado por voltagem positiva e um bit 0 por voltagem negativa. Em fibras ópticas, essa representação se traduz na presença de luz para o bit 1 e na ausência de luz para o bit 0. Essa codificação é direta, onde o sinal transmitido corresponda exatamente aos dados originais. A codificação (c) NRZI é basicamente a NRZ com as representações invertidas.

A codificação (d) Manchester representa os bits através de transições de tensão no meio do período de bit. Cada slot vertical na figura representa um período de bit. O bit 0 é representado como uma transição de baixo para alto enquanto o bit 1 é representado como transição de alto para baixo. (*Obs. Alguns autores apresentam o bit 0 como sendo a transição de alto para baixo e o bit 1 como sendo de baixo para alto*).

Exemplo - Código Morse: O código Morse é um sistema de codificação de caracteres que utiliza combinações de sinais curtos (pontos) e longos (traços) para representar letras, números e símbolos. Foi desenvolvido na década de 1830 para telegrafia e cada caractere possui uma sequência única de pontos e traços (Figura 6.2). Espaços curtos separam os sinais de uma mesma letra e espaços mais longos entre palavras. Uma mensagem codificada em Morse pode ser transmitida de várias maneiras: som, luz ou pulsos elétricos.

Sinalização da mensagem

Sinalização é a representação dos bits 1 e 0 através das características do meio físico usado para a transmissão. Os padrões da camada física especificam qual tipo de sinal corresponde aos bits 1 e 0. A camada física é responsável por gerar os sinais elétricos, ópticos ou ondas eletromagnéticas para representar o código usado na codificação dos bits de transmissão.

Em um fio de cobre, a sinalização dos bits 0 e 1 pode ser feita através de variações da voltagem ao longo do tempo. Em uma fibra óptica, a sinalização dos bits 0 e 1 pode ser feita através de pulsos de luz. E, em uma transmissão sem fio, uma onda eletromagnética pode sinalizar os bits 0 e 1 através

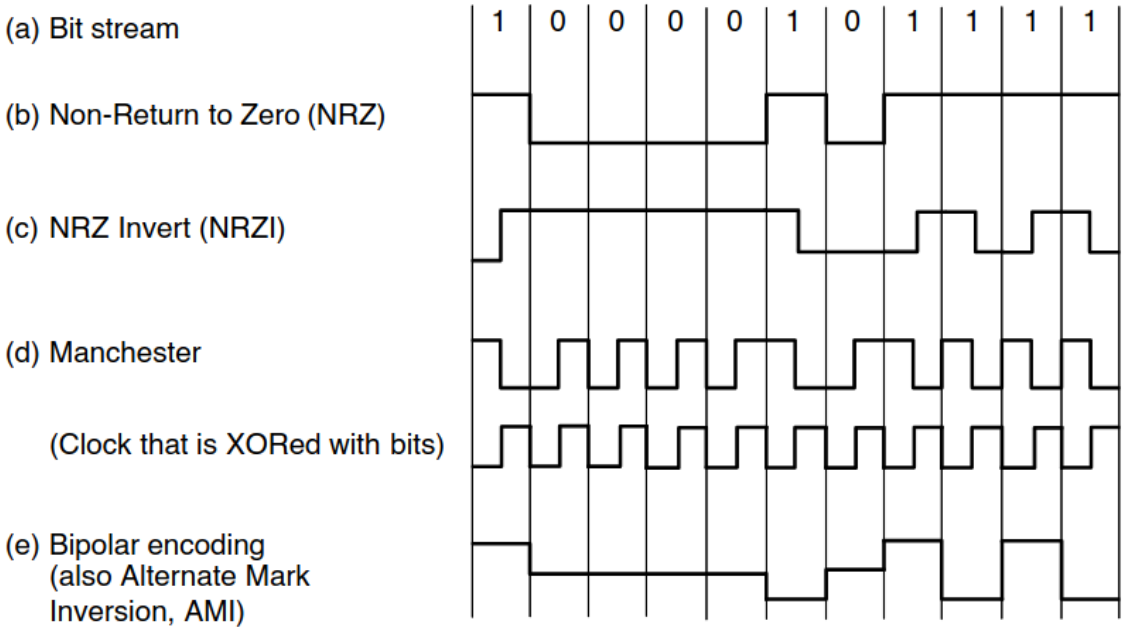

Figure 2-14. Line codes: (a) Bits, (b) NRZ, (c) NRZI, (d) Manchester, (e) Bipolar or AMI.

Figura 6.1. Tipos de código - *Computer Networks [?]*

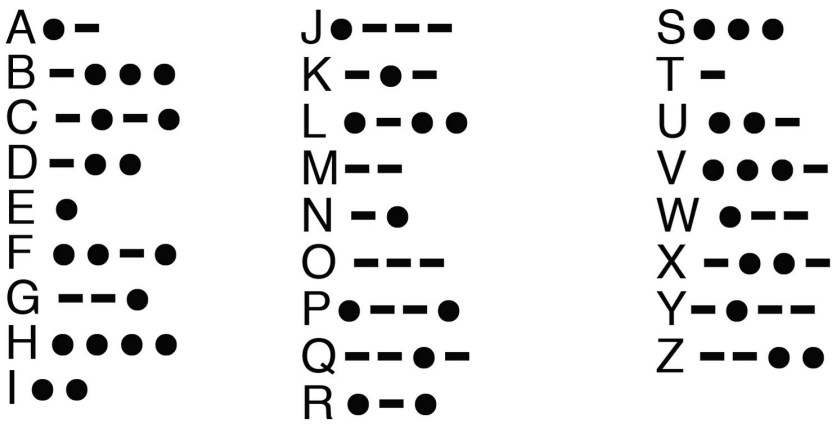

Figura 6.2. Tabela de código Morse - Ref: https://www.infoescola.com/wp-content/uploads/2013/02/codigo-morse.jpg

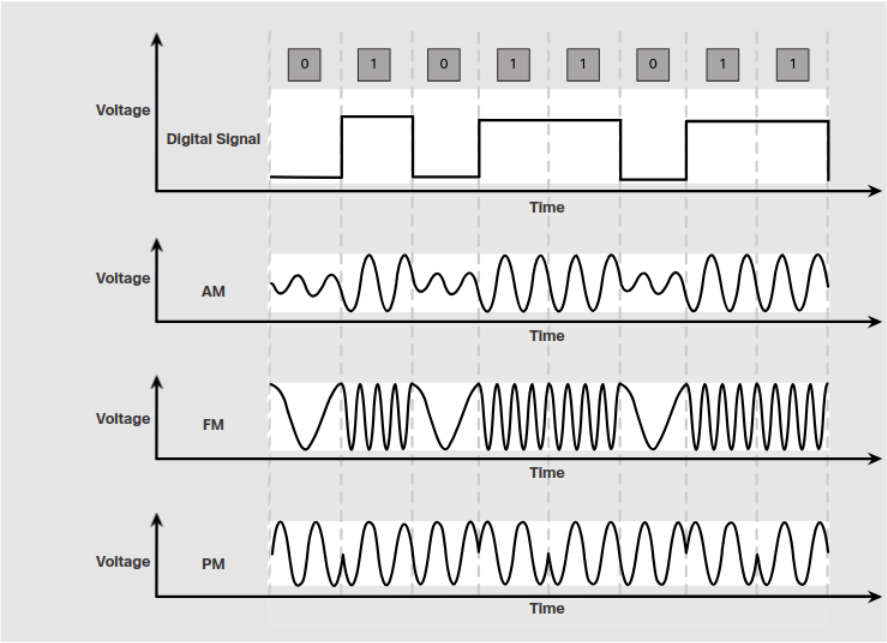

Figura 6.3. Sinalização em uma onda eletromagnética [ref cisco netacad]

da amplitude, frequência ou fase da onda (Figura 6.3).

Camada de Enlace

A camada de enlace está preocupada com como podemos enviar mensagens entre dispositivos diretamente conectados e como podemos acessar o meio físico ao qual esses dispositivos estão ligados.

6.1.2. Camada Internet

A camada Internet é onde toda essa arquitetura se integra. Basicamente, o seu objetivo é permitir que dispositivos sejam capazes de colocar pacotes em qualquer rede, de forma que eles possam viajar independentemente até o seu destino, que pode estar em outra rede. Esses pacotes podem chegar em uma ordem completamente diferente daquela em que foram enviados.

6.1.3. Camada de Transporte

A camada de transporte fica acima da camada de Internet. O seu desafio é permitir que as entidades na origem e no destino — também chamadas de processos — sejam capazes de carregar uma conversa, definindo como será o transporte das informações. Esse transporte pode ser realizado de duas maneiras: com o estabelecimento de uma conexão, possibilitando o controle da ordem e do fluxo dos pacotes, bem como a garantia de uma transmissão confiável; ou de forma não confiável, na qual não há necessidade de controlar a ordem, o fluxo, nem de garantir a entrega dos dados.

6.1.4. Camada de Aplicação

A camada de aplicação é a camada de mais alto nível do modelo. Ela está em contato direto com o usuário e oferece aplicações que utilizam a rede para se conectar com outras aplicações, que podem estar sendo usadas por outras pessoas, potencialmente em outras redes.

As primeiras aplicações foram terminais virtuais, como o Telnet, transferência de arquivos, como o FTP, ou o correio eletrônico, utilizando o SMTP. Desde então, muitos outros protocolos de aplicação foram desenvolvidos e, atualmente, temos a Web e o HTTP como as principais aplicações utilizadas mundialmente.

6.2. Estrutura conceitual

A comunicação de dados envolve essencialmente três áreas de conhecimento: (i) **Física** (a transmissão de dados se dá por meios físicos, usando corrente elétrica, feixes de luz, ondas de rádio e outras formas de radiação eletromagnética); (ii) **Matemática** (teoria da informação e formas de análises); e (iii) **Engenharia elétrica e de telecomunicações** (alternativas práticas de projetar e construir sistemas de transmissão de dados) [Comer 2016].

Os fundamentos da comunicação de dados se estruturam a partir de três considerações/princípios:

(1) **as fontes de informação são de formas variadas** (ex., bits de um computador, áudio de um microfone, vídeo de uma câmera) e é possível transformar uma forma em outra;
(2) **a transmissão utiliza um meio físico**, então é preciso compreender como fenômenos físicos podem ser usados para transmitir dados, quais são os meios disponíveis e suas propriedades, quais são os limites dos meios físicos e as técnicas usadas para detectar e resolver problemas;
(3) **as várias fontes de dados podem compartilhar o meio subjacente**, o seja, vários pares de comunicação podem ser estabelecidos em um mesmo meio físico (conhecer vantages e desvantages das diferentes formas de compartilhamento).

A Figura 6.4 apresenta um estrutura conceitual de um sistema de comunicação de dados, onde várias fontes enviam dados para vários destinos por meio de um canal multiplexado.

As **fontes de informação** podem ser analógicas ou digitais e podem ser convertidas de uma forma para outra. Os **codificadores/decodificadores de fonte** permitem transformar e converter as representações digitais, por exemplo, comprimindo os dados. Os **encriptadores/decriptadores** são usados para manter o dados confidenciais. Os **codificadores/decodificadores de canal** são usados para tratar (detectar e corrigir) erros de transmissão dos bits. Os **multiplexadores/demultiplexadores** definem formas de combinar as diferentes fontes de dados em um único meio (compartilhamento simultâneo ou com revezamento). Os **moduladores/demoduladores** tratam como usar o fenômeno físico para enviar e receber dados (esquemas de modulação analógica e digital). Por fim, o **canal físico** define meios e modos de transmissão [Comer 2016].

6.3. Fontes de dados e sinais

Dados podem ser representados por sinais: **analógicos** ou **digitais** [Comer 2016]:

- **Analógico**: função matemática contínua, ou seja, quando a entrada muda de um valor para o próximo, faz isso movendo-se através de todos os valores intermediários (ex., fala humana, onda eletromagnética continuamente variável, que pode ser transmitida em diferentes meios).
- **Digital**: conjunto fixo (e finito) de níveis válidos e cada mudança consiste em uma transição instantânea de um nível válido para outro. A intensidade do sinal mantém um nível constante por um tempo e depois muda para outro nível (ex., binários 1 e 0, sequência de pulsos de voltagem que podem ser transmitidos em meios de cobre).

A Figura 6.5 ilustra um sinal analógico, simples (não pode ser decomposto em outros sinais) e periódico (se repete em intervalos de tempo). Três características estão associadas a esse sinal: amplitude (valor máximo e mínimo), fase (deslocamento à direita ou à esquerda) e frequência (ciclos por segundo). Sinais simples como o da Fugura 6.5 podem ser agregados e formar **sinais compostos**.

O matemático Fourier demonstrou que sinais compostos e periódicos podem ser decompostos em um conjunto de sinais simples (ondas senoidais), cada um com frequência, amplitude e fase próprios. O método matemático proposto por Fourier permite aproximar funções quadradas — como aquelas que representam dados digitais — em uma função analógica resultante da composição de várias funções senoidais simples. Parte dos sistemas de comunicação usam esse método para transmitir dados: um sinal composto é criado no envio (do lado da fonte do dado) e decomposto na recepção (do lado do destino do dado).

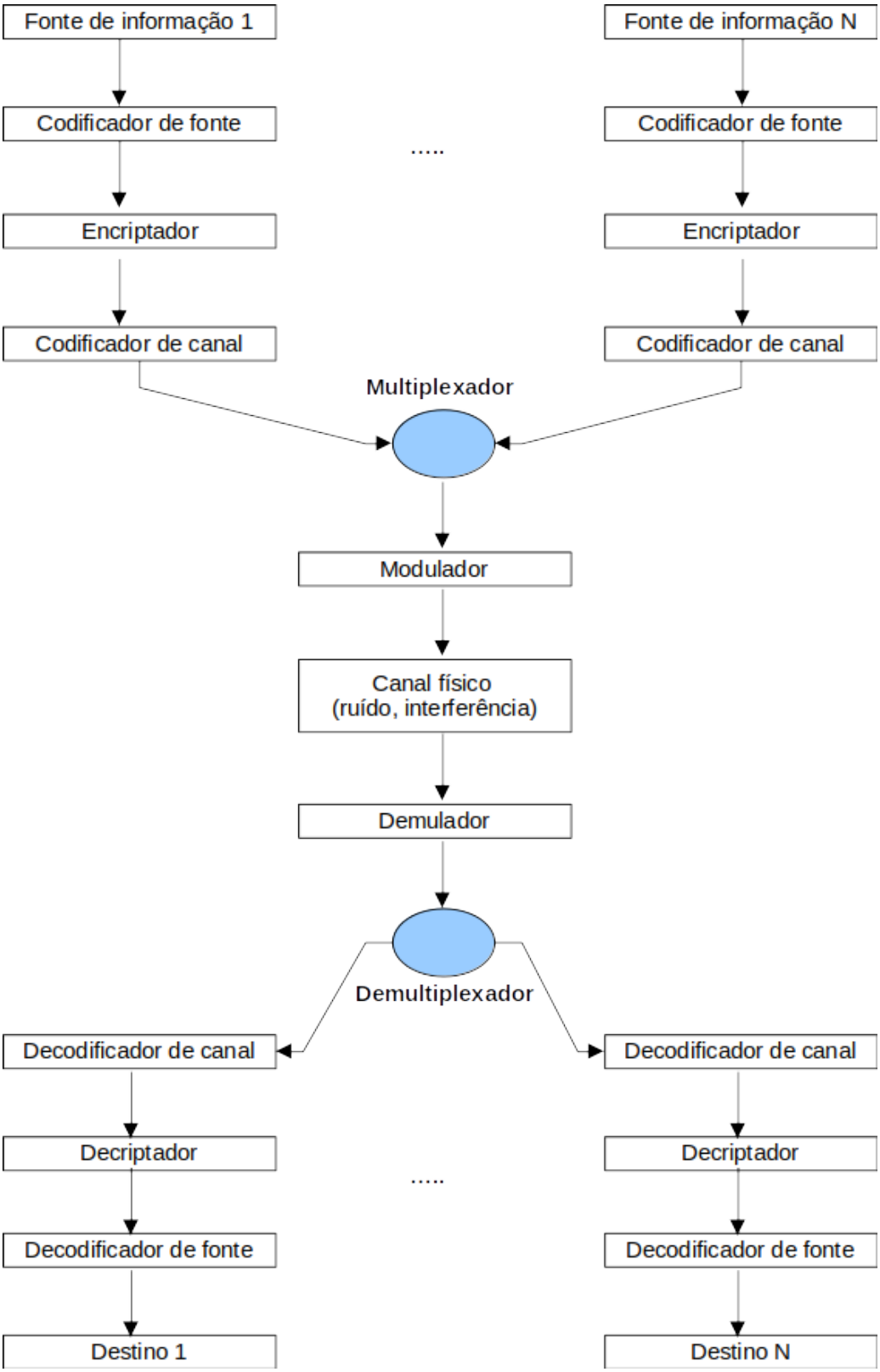

Figura 6.4. Visão em camada de um sistema de computação [Comer 2016].

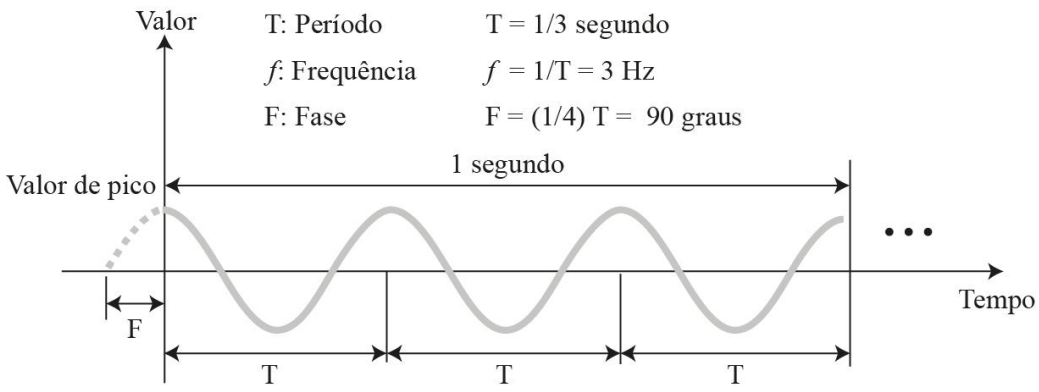

Figura 6.5. Sinal analógico simples e periódico.

A Figura 6.6 ilustra a aproximação de um sinal digital para um sinal analógico, usando o método proposto por Fourier. Cada onda senoidal simples é chamada *harmônica* e sua frequência é definida como sendo múltipla da frequência fundamental (frequência do sinal original). Daí chegamos ao conceito de **largura de banda de um sinal analógico**: diferença entre a frequência mais alta e a frequência mais baixa das componentes constituintes do sinal. A segunda coluna da Figura 6.6 mostra exatamente a representação do sinal analógico no **domínio das frequências** (permite uma representação mais compacta das mesmas informações). No eixo x representamos as frequências (na forma de múltiplas da frequência fundamental) e no eixo y a amplitude do sinal correspondente.

Exercícios 1. Com base no que foi exposto, escreva com suas palavras:

1. Por que a comunicação de dados envolve Matemática, Física e Engenharia Elétrica? O que de cada área é usado para construir sistemas de comunicação?
2. Quais são os princípios fundamentais da comunicação de dados?

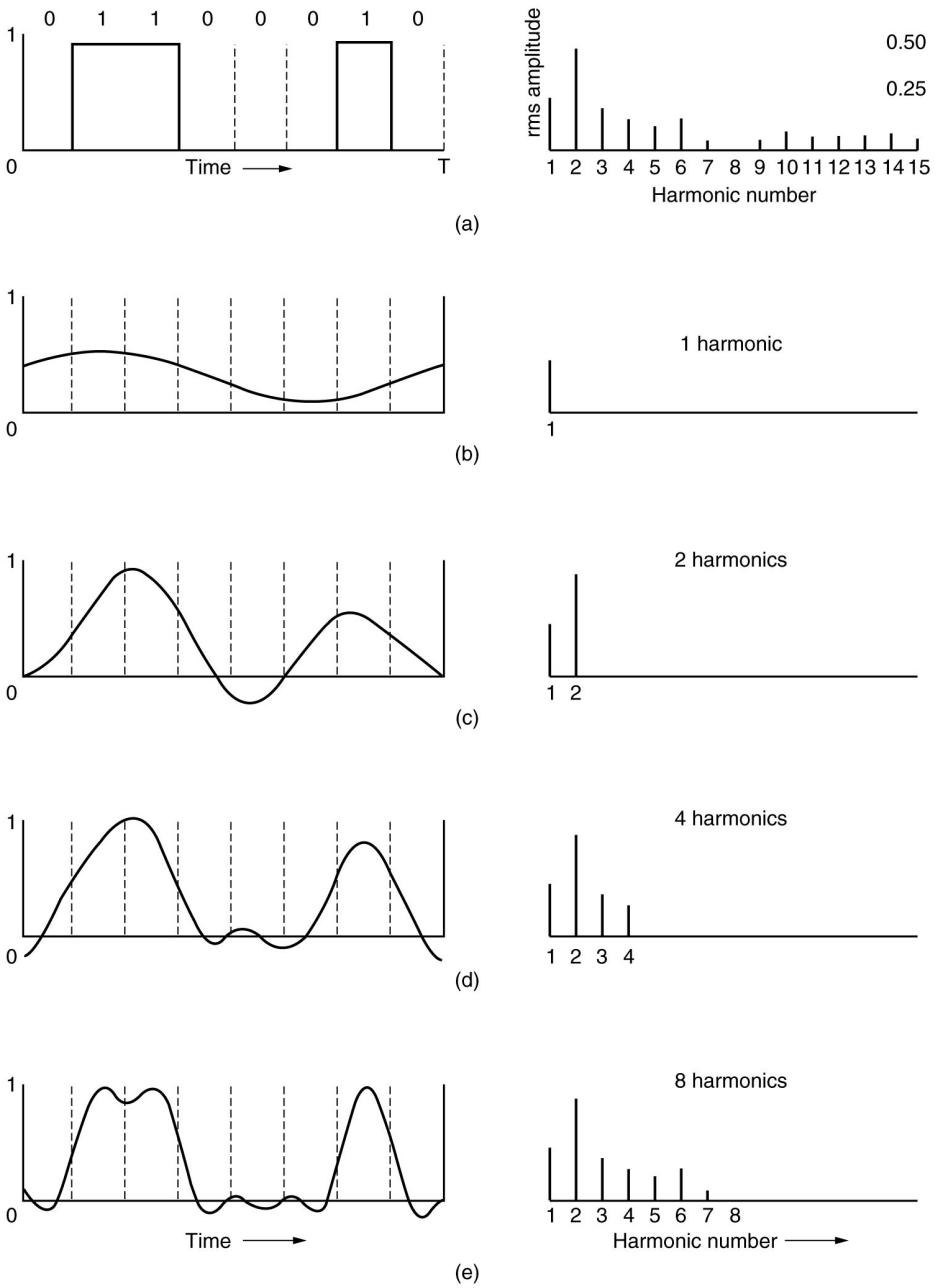

Figura 6.6. Exemplo ilustrativo de aproximação de um sinal digital para analógico.

Referências

Bryant, R. E. and O'Hallaron, D. R. (2016). *Computer systems: a programmer's perspective*. Pearson, 3 edition.

Comer, D. E. (2016). *Redes de Computadores e Internet*. Bookman, 6 edition.

Tanenbaum, A. (1992). *S. Organização estruturada de computadores*. Prentice Hall Brasil, 3 edition.

Wikipedia (2025). Character encoding.